Predictive Analytics for Credit Risk Management: Enhancing Default Prediction in Banking

Capstone Project Report

Data Science and Business Analytics

Great Learning

Name: Sattaru Manohar

Batch: PGPDSBA.O. APR23.A

Contents

Content	Page Number
1. Introduction	3
1 a. Brief introduction about the problem statement and the need of solving it	3
2. EDA and Business Implication	4 - 12
2 a. Uni-variate / Bi-variate / multi-variate analysis to understand relationship b/w variables. How your analysis is impacting the business?	4 - 9
2 b. Both visual and non-visual understanding of the data	10 - 12
3. Data Cleaning and Pre-processing	13 - 16
3 a. Approach used for identifying and treating missing values and outlier treatment (and why)	13 - 15
3 b. Need for variable transformation (if any)	15
3 c. Variables removed or added and why (if any)	15 - 16
4. Model building	16 - 24
4 a. Clear on why was a particular model(s) chosen and improve model performance.	
5. Model validation	24 - 28
5 a. How was the model validated? Just accuracy, or anything else too?	
6. Final interpretation / recommendation	28 - 30
6 a. Detailed recommendations for the management/client based on the analysis done.	

S. No	List of Figures	Page No
1	Figure 1: Histogram plots for numerical variables	4
2	Figure 2: Histogram plot for distribution of has_paid variable	5
3	Figure 3: Histogram plot for distribution of merchant group variable	5
4	Figure 4: Histogram plot for distribution of merchant category variable	6
5	Figure 5: Correlation heatmap for all variables	7
6	Figure 6: Pair plot for continuous attributes	8
7	Figure 7: Boxplots for all variables vs default with outliers	9
8	Figure 8: Missing values representation in heat map	14
9	Figure 9: Outlier representation in boxplots for numerical variables	14
10	Figure 10: Train - Confusion Matrix – Logistic regression	17
11	Figure 11: Train - (ROC) Curve – Logistic regression	18
12	Figure 12: Train - Precision-Recall Curve – Logistic regression	18
13	Figure 13: Train - Confusion Matrix – Decision Tree	18
14	Figure 14: Train - (ROC) Curve – Decision tree	18
15	Figure 15: Train - Precision-Recall Curve – Decision Tree	18
16	Figure 16: Train - Confusion Matrix – Random Forest	19
17	Figure 17: Train - (ROC) Curve – Random Forest	19
18	Figure 18: Train - Precision-Recall Curve – Random Forest	19
19	Figure 19: Train - Confusion Matrix – AdaBoost	19
20	Figure 20: Train - (ROC) Curve – AdaBoost	20
21	Figure 21: Train - Precision-Recall Curve – AdaBoost	20
22	Figure 22: Train - Confusion Matrix – Gradient Boosting	20
23	Figure 23: Train - (ROC) Curve – Gradient Boosting	20
24	Figure 24: Train - Precision-Recall Curve – Gradient Boosting	20
25	Figure 25: Train - Confusion Matrix – Bagging	21
26	Figure 26: Train - (ROC) Curve – Bagging	21
27	Figure 27: Train - Precision-Recall Curve – Bagging	21
28	Figure 28: Train - Confusion Matrix – Light GBM	21
29	Figure 29: Train - (ROC) Curve – Light GBM	22
30	Figure 30: Train - Precision-Recall Curve – Light GBM	22
31	Figure 31: Train - Confusion Matrix – XG Boost	22
32	Figure 32: Train - (ROC) Curve – XG Boost	22
33	Figure 33: Train - Precision-Recall Curve – XG Boost	22
34	Figure 34: Train - Confusion Matrix – LDA	23
35	Figure 35: Train - (ROC) Curve – XG Boost	23
36	Figure 36: Train - Precision-Recall Curve – XG Boost	23
37	Figure 37: Train summary – Combined (ROC) Curve for all models	23
38	Figure 38: Training summary – Combined Precision-Recall Curve for all models	24
39	Figure 39: Test summary – Combined (ROC) Curve for all models	27
40	Figure 40: Test summary – Combined Precision-Recall Curve for all models	27
41	Figure 41: Test accuracy comparison for all models	28
1	Table 1: Model comparison – Train and Test Accuracies	28

1. Introduction to the business problem

1a. Brief introduction about the problem statement and the need of solving it

Sol: The problem statement is to build a supervised learning classification model for a credit card company, focusing on predicting the probability of default. The objective is to predict whether a customer will pay their credit card bill or default based on the provided variables in the dataset and predictive modelling task is very important in understanding the riskiness of customers and analysing the potential credit exposure in case of default.

Default prediction models play an integral role in institutions that deal with finances especially those lending money since they help in analysing clients' creditworthiness. By leveraging various account-related, purchase, and delinquency information, these models enable organizations to manage risk effectively and make informed decisions regarding credit allocation.

The aim is to develop a robust predictive classification model that enhances the company's ability to identify and mitigate potential default risks, thereby safeguarding its financial health and ensuring prudent lending practices.

The need for study/project is essential because predictive modeling plays a significant role in the operations of financial institutions, especially those in the lending sector. It helps the client to understand and manage default risks within the credit card banking industry. By accurately predicting the likelihood of customers defaulting on their credit card payments, the company can be protected from potential financial losses further helps in maintaining the financial stability of the company.

1. Understanding Riskiness: The Predictive modeling enables us to explore the behaviours and patterns of customers, which helps us to assess the level of riskiness. This understanding is essential for identifying potential defaulters and managing risk effectively from the short term and long-term financial losses.

2. Analyzing Credit Exposure: By predicting the probability of default, we can gain insights into the potential credit exposure faced by the company. This analysis is crucial for assessing the financial impact of defaulting customers and strategizing risk mitigation measures.

3. Enhancing Decision-making: Default prediction models serve as powerful tools for making informed decisions regarding credit allocation. By leveraging various account-related, purchase, and delinquency information, these models enable organizations to allocate credit resources wisely, maximizing profitability while minimizing risk.

4. Promoting Financial Health: A robust predictive classification model not only safeguards the financial health of the company but also promotes responsible lending practices. By accurately identifying and mitigating potential default risks, the company can uphold its reputation and credibility in the financial market.

In summary, the study/project addresses a critical need within the financial sector, offering a proactive approach to risk management and ensuring the sustainability and growth of the company in the long run.

2. EDA and Business Implication

2 a. Uni-variate / Bi-variate / multi-variate analysis to understand relationship b/w variables. How your analysis is impacting the business?

Sol: In this project, the data was collected from a credit card company's database. The dataset contains information about various aspects of credit card accounts, including purchase behaviours, delinquency status, account activity, and demographic details of the customers.

- Shape of the dataset: **(Rows - 99979, Columns - 36)]**

1. Univariate /Bivariate/ Multi variate analysis of numerical and categorical variables:

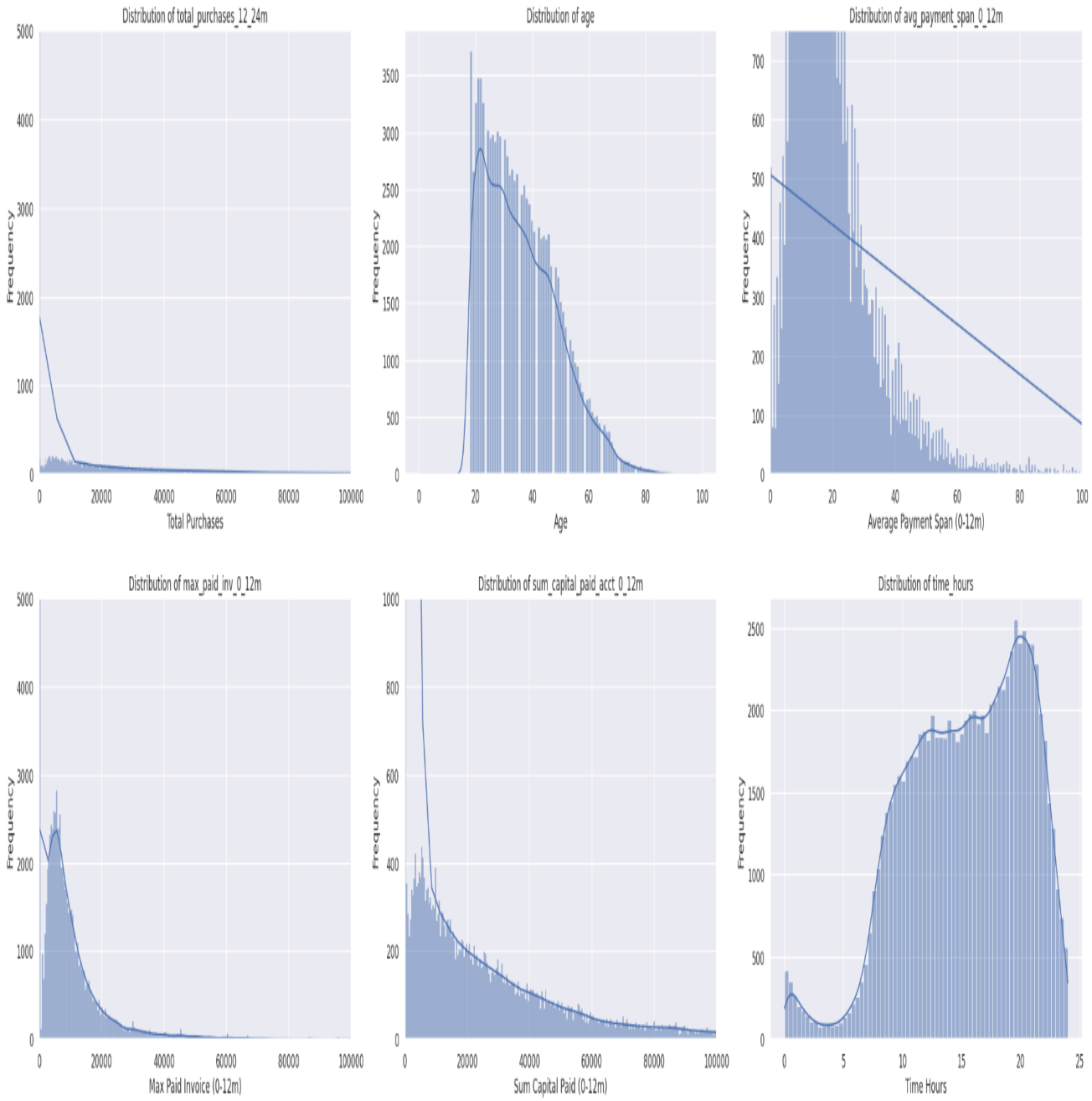

Figure 1: Histogram plots for numerical variables

Univariate analysis for Categorical variables: Has_paid, Merchant group and Merchant category

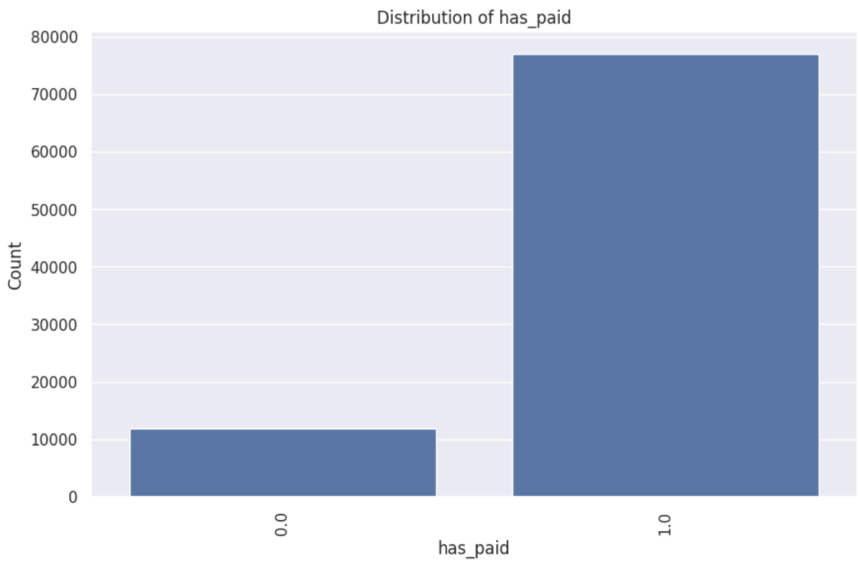

Figure 2: Histogram plot for distribution of has_paid variable

From the data it has been observed that majority of the customers paid the credit card bill on time

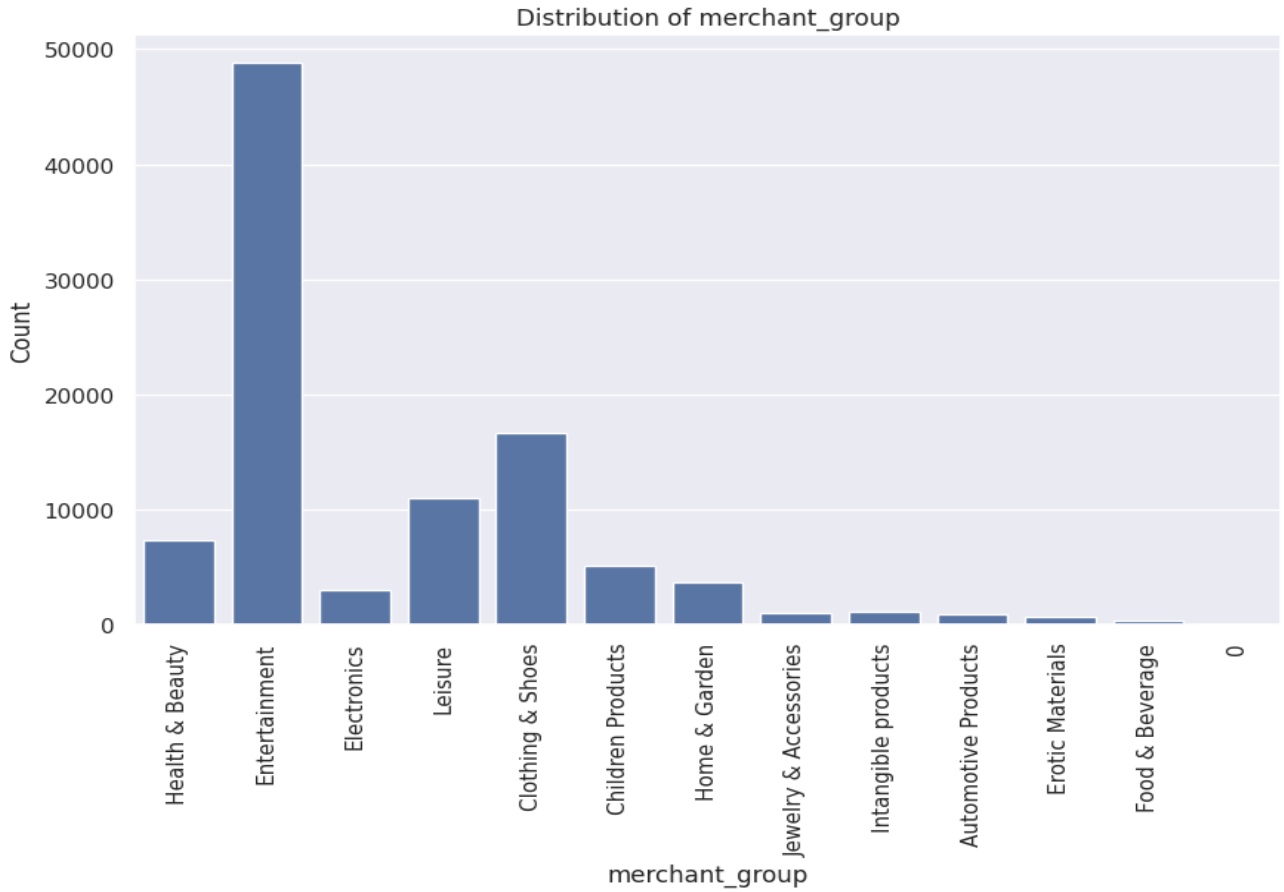

Figure 3: Histogram plot for distribution of merchant group variable

From the above graph most of the credit spending on entertainment and followed by clothing and shoes

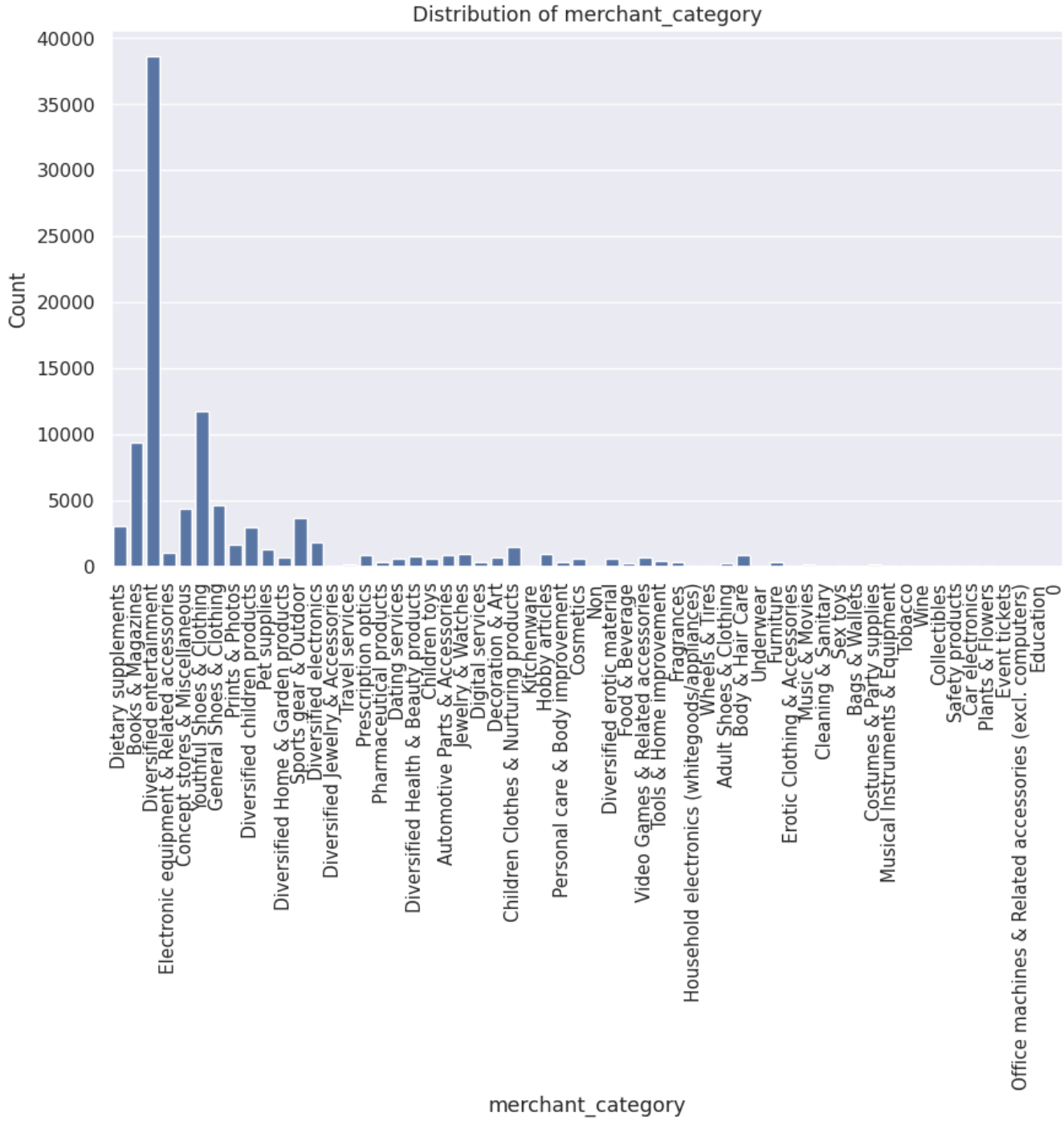

Figure 4: Histogram plot for distribution of merchant category variable

Quick summary of all the above graphs

- Fig1 - Age variable in which the distribution is lies in between 20 – 40 age group and average payment span or day is in between 0 to 40 days.
- Fig3 – Majority in Entertainment followed by clothing & shoes category in the merchant group variable
- Fig4 – Majority of the merchant lies in diversified entertainment

Correlation matrix for variables:

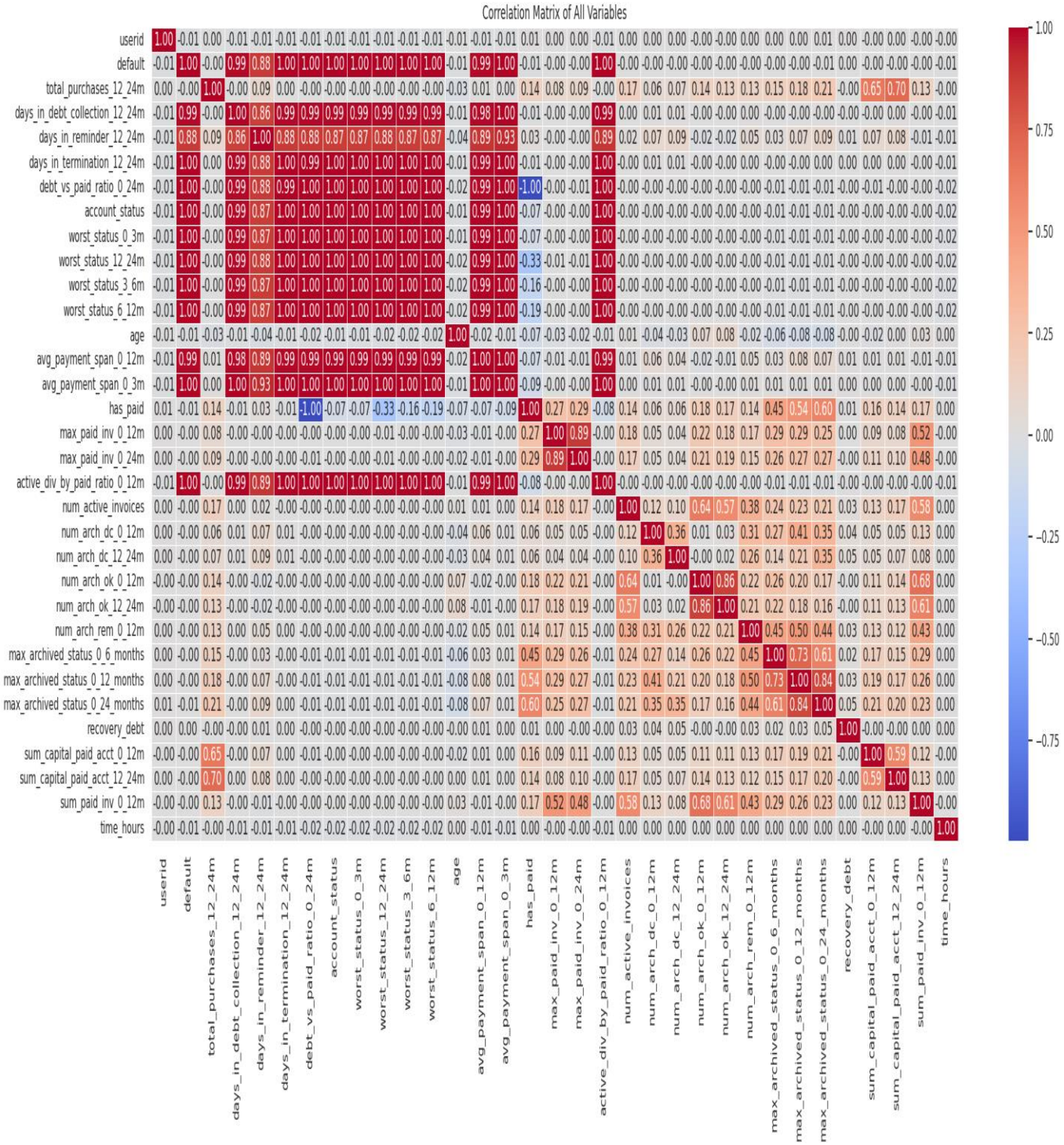

Figure 5: Correlation heatmap for all variables

1. Variable has_paid has the negative correlation with debt_vs_paid_ratio_0_24m, worst_status_12_24m, worst_status_3_6m, worst_status_6_12m.
2. Variable default has a strong positive correlation with the avg_payment_span, account status, days_in_default_collection

Pair plots for continuous attributes:

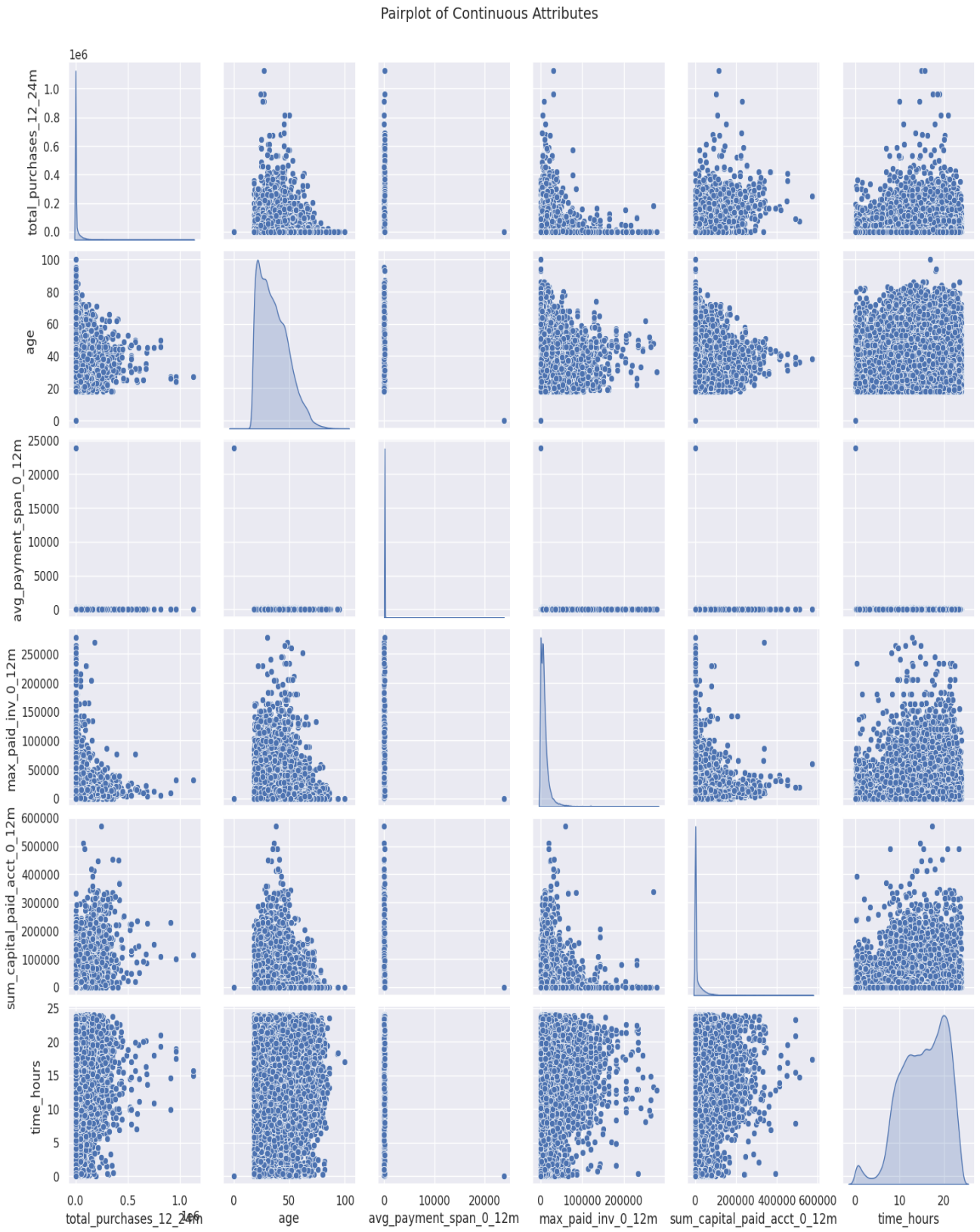

Figure 6: Pair plot for continuous attributes

In the above pair plot it has been observed that there are some relationships with age and total purchases, max_paid_inv_0_12m.

Analysis for all the variables with default:

Figure 7: Boxplots for all variables vs default with outliers

2 b. Both visual and non-visual understanding of the data

Sol: For the visual inspection of data, we need to examine the dataset provided for our credit card default prediction problem. Let us start by loading the dataset and exploring its structure, including the rows, columns, and descriptive details of each column.

- Description about the dataset:

	count	mean	std	min	25%	50%	75%	max
userid	99977.0	2.998947e+06	1.154211e+06	0.0	2.000260e+06	2.998815e+06	4.000633e+06	4.999868e+06
default	89977.0	1.254543e-01	3.333776e+01	0.0	0.000000e+00	0.000000e+00	0.000000e+00	1.000000e+04
acct_amt_added_12_24m	99977.0	1.225503e+04	3.548133e+04	0.0	0.000000e+00	0.000000e+00	4.937000e+03	1.128775e+06
acct_days_in_dc_12_24m	88141.0	3.573252e-01	4.028733e+01	0.0	0.000000e+00	0.000000e+00	0.000000e+00	1.183600e+04
acct_days_in_rem_12_24m	88141.0	5.178850e+00	4.594340e+01	0.0	0.000000e+00	0.000000e+00	0.000000e+00	1.183600e+04
acct_days_in_term_12_24m	88141.0	4.211774e-01	3.997377e+01	0.0	0.000000e+00	0.000000e+00	0.000000e+00	1.183600e+04
acct_incoming_debt_vs_paid_0_24m	40662.0	2.789992e+00	2.953340e+02	0.0	0.000000e+00	1.520899e-01	6.629926e-01	5.931500e+04
acct_status	45604.0	2.234431e+00	2.546089e+02	1.0	1.000000e+00	1.000000e+00	1.000000e+00	5.437300e+04
acct_worst_status_0_3m	45604.0	2.365165e+00	2.546086e+02	1.0	1.000000e+00	1.000000e+00	1.000000e+00	5.437300e+04
acct_worst_status_12_24m	33216.0	3.347212e+00	3.663034e+02	1.0	1.000000e+00	1.000000e+00	2.000000e+00	6.676100e+04
acct_worst_status_3_6m	42275.0	2.550183e+00	2.806343e+02	1.0	1.000000e+00	1.000000e+00	1.000000e+00	5.770200e+04
acct_worst_status_6_12m	39627.0	2.776062e+00	3.031610e+02	1.0	1.000000e+00	1.000000e+00	1.000000e+00	6.035000e+04
age	99977.0	3.601592e+01	1.300174e+01	0.0	2.500000e+01	3.400000e+01	4.500000e+01	1.000000e+02
avg_payment_span_0_12m	76141.0	1.828429e+01	8.725376e+01	0.0	1.080000e+01	1.490909e+01	2.100000e+01	2.383600e+04
avg_payment_span_0_3m	50672.0	1.596251e+01	2.192071e+02	0.0	8.400000e+00	1.300000e+01	1.828571e+01	4.930500e+04
has_paid	88943.0	8.657455e-01	3.409275e-01	0.0	1.000000e+00	1.000000e+00	1.000000e+00	1.000000e+00
max_paid_inv_0_12m	88943.0	9.362712e+03	1.367242e+04	0.0	2.387500e+03	6.170000e+03	1.140000e+04	2.790000e+05
max_paid_inv_0_24m	88943.0	1.141961e+04	1.543171e+04	0.0	3.685000e+03	7.720000e+03	1.386500e+04	5.385000e+05
num_active_div_by_paid_inv_0_12m	70052.0	3.818166e-01	7.137492e+01	0.0	0.000000e+00	0.000000e+00	1.000000e-01	1.889100e+04
num_active_inv	88943.0	6.259852e-01	1.610529e+00	0.0	0.000000e+00	0.000000e+00	1.000000e+00	4.700000e+01
num_arch_dc_0_12m	88943.0	6.261313e-02	3.821109e-01	0.0	0.000000e+00	0.000000e+00	0.000000e+00	1.700000e+01
num_arch_dc_12_24m	88943.0	5.922894e-02	3.623208e-01	0.0	0.000000e+00	0.000000e+00	0.000000e+00	1.300000e+01
num_arch_ok_0_12m	88943.0	7.786448e+00	1.674280e+01	0.0	0.000000e+00	3.000000e+00	8.000000e+00	2.610000e+02
num_arch_ok_12_24m	88943.0	6.846576e+00	1.606794e+01	0.0	0.000000e+00	2.000000e+00	7.000000e+00	3.130000e+02
num_arch_rem_0_12m	88943.0	4.839954e-01	1.395611e+00	0.0	0.000000e+00	0.000000e+00	0.000000e+00	4.200000e+01
status_max_archived_0_6_months	88943.0	8.217398e-01	7.166438e-01	0.0	0.000000e+00	1.000000e+00	1.000000e+00	3.000000e+00
status_max_archived_0_12_months	88943.0	1.074182e+00	7.763897e-01	0.0	1.000000e+00	1.000000e+00	2.000000e+00	5.000000e+00
status_max_archived_0_24_months	88943.0	1.248193e+00	8.205178e-01	0.0	1.000000e+00	1.000000e+00	2.000000e+00	5.000000e+00
recovery_debt	88943.0	3.601925e+00	1.162108e+02	0.0	0.000000e+00	0.000000e+00	0.000000e+00	1.641100e+04
sum_capital_paid_acct_0_12m	88943.0	1.086026e+04	2.663062e+04	0.0	0.000000e+00	0.000000e+00	8.959500e+03	5.714750e+05
sum_capital_paid_acct_12_24m	88943.0	6.614946e+03	1.924381e+04	0.0	0.000000e+00	0.000000e+00	1.025000e+02	3.418590e+05
sum_paid_inv_0_12m	88943.0	4.103591e+04	9.459642e+04	0.0	3.395500e+03	1.705700e+04	4.573900e+04	2.962870e+06
time_hours	88943.0	1.534165e+01	5.030877e+00	0.0	1.163181e+01	1.580833e+01	1.955417e+01	2.399972e+01

To understand the attributes of the dataset and perform any renaming, we will follow these steps:

- Information and summary of the provided dataset.

In the below output it has been observed that there are lot of data points missing in the dataset.

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 99979 entries, 0 to 99978
Data columns (total 36 columns):
 #   Column                            Non-Null Count  Dtype
---  ------                            --------------  -----
 0   userid                            99977 non-null  float64
 1   default                           89977 non-null  float64
 2   acct_amt_added_12_24m             99977 non-null  float64
 3   acct_days_in_dc_12_24m            88141 non-null  float64
 4   acct_days_in_rem_12_24m           88141 non-null  float64
 5   acct_days_in_term_12_24m          88141 non-null  float64
 6   acct_incoming_debt_vs_paid_0_24m  40662 non-null  float64
 7   acct_status                       45604 non-null  float64
 8   acct_worst_status_0_3m            45604 non-null  float64
 9   acct_worst_status_12_24m          33216 non-null  float64
 10  acct_worst_status_3_6m            42275 non-null  float64
 11  acct_worst_status_6_12m           39627 non-null  float64
 12  age                               99977 non-null  float64
 13  avg_payment_span_0_12m            76141 non-null  float64
 14  avg_payment_span_0_3m             50672 non-null  float64
 15  merchant_category                 99977 non-null  object
 16  merchant_group                    99968 non-null  object
 17  has_paid                          88943 non-null  float64
 18  max_paid_inv_0_12m                88943 non-null  float64
 19  max_paid_inv_0_24m                88943 non-null  float64
 20  name_in_email                     88943 non-null  object
 21  num_active_div_by_paid_inv_0_12m  70052 non-null  float64
 22  num_active_inv                    88943 non-null  float64
 23  num_arch_dc_0_12m                 88943 non-null  float64
 24  num_arch_dc_12_24m                88943 non-null  float64
 25  num_arch_ok_0_12m                 88943 non-null  float64
 26  num_arch_ok_12_24m                88943 non-null  float64
 27  num_arch_rem_0_12m                88943 non-null  float64
 28  status_max_archived_0_6_months    88943 non-null  float64
 29  status_max_archived_0_12_months   88943 non-null  float64
 30  status_max_archived_0_24_months   88943 non-null  float64
 31  recovery_debt                     88943 non-null  float64
 32  sum_capital_paid_acct_0_12m       88943 non-null  float64
 33  sum_capital_paid_acct_12_24m      88943 non-null  float64
 34  sum_paid_inv_0_12m                88943 non-null  float64
 35  time_hours                        88943 non-null  float64
dtypes: float64(33), object(3)
memory usage: 27.5+ MB
```

- Check the data types of each column to ensure they are appropriate.

```
userid                              float64
default                             float64
acct_amt_added_12_24m               float64
acct_days_in_dc_12_24m              float64
acct_days_in_rem_12_24m             float64
acct_days_in_term_12_24m            float64
acct_incoming_debt_vs_paid_0_24m    float64
acct_status                         float64
acct_worst_status_0_3m              float64
acct_worst_status_12_24m            float64
acct_worst_status_3_6m              float64
acct_worst_status_6_12m             float64
age                                 float64
avg_payment_span_0_12m              float64
avg_payment_span_0_3m               float64
merchant_category                    object
merchant_group                       object
has_paid                            float64
max_paid_inv_0_12m                  float64
max_paid_inv_0_24m                  float64
name_in_email                        object
num_active_div_by_paid_inv_0_12m    float64
num_active_inv                      float64
num_arch_dc_0_12m                   float64
num_arch_dc_12_24m                  float64
num_arch_ok_0_12m                   float64
num_arch_ok_12_24m                  float64
num_arch_rem_0_12m                  float64
status_max_archived_0_6_months      float64
status_max_archived_0_12_months     float64
status_max_archived_0_24_months     float64
recovery_debt                       float64
sum_capital_paid_acct_0_12m         float64
sum_capital_paid_acct_12_24m        float64
sum_paid_inv_0_12m                  float64
time_hours                          float64
dtype: object
```

There are 33 variables with float type and 3 variables with object type in the provided dataset.

3. Data Cleaning and Pre-processing

3 a. Approach used for identifying and treating missing values and outlier treatment (and why)

Sol: Approach used to identify the missing values by detecting the null values in the provided dataset.

```
Missing Data Count in Ascending Order:
total_purchases_12_24m                 2
merchant_category                      2
age                                    2
merchant_group                        11
default                            10002
num_arch_dc_0_12m                  11036
num_arch_dc_12_24m                 11036
num_arch_ok_0_12m                  11036
num_arch_ok_12_24m                 11036
num_arch_rem_0_12m                 11036
max_archived_status_0_6_months     11036
max_archived_status_0_12_months    11036
max_archived_status_0_24_months    11036
recovery_debt                      11036
sum_capital_paid_acct_0_12m        11036
sum_capital_paid_acct_12_24m       11036
num_active_invoices                11036
name_in_email                      11036
max_paid_inv_0_12m                 11036
sum_paid_inv_0_12m                 11036
has_paid                           11036
max_paid_inv_0_24m                 11036
time_hours                         11036
days_in_termination_12_24m         11838
days_in_reminder_12_24m            11838
days_in_debt_collection_12_24m     11838
avg_payment_span_0_12m             23838
active_div_by_paid_ratio_0_12m     29927
avg_payment_span_0_3m              49307
worst_status_0_3m                  54375
account_status                     54375
worst_status_3_6m                  57704
debt_vs_paid_ratio_0_24m           59317
worst_status_6_12m                 60352
worst_status_12_24m                66763
```

```
Percentage of Missing Values for Each Column:
default                            10.004101
total_purchases_12_24m              0.002000
days_in_debt_collection_12_24m     11.840487
days_in_reminder_12_24m            11.840487
days_in_termination_12_24m         11.840487
debt_vs_paid_ratio_0_24m           59.329459
account_status                     54.386421
worst_status_0_3m                  54.386421
worst_status_12_24m                66.777023
worst_status_3_6m                  57.716120
worst_status_6_12m                 60.364677
age                                 0.002000
avg_payment_span_0_12m             23.843007
avg_payment_span_0_3m              49.317357
merchant_category                   0.002000
merchant_group                      0.011002
has_paid                           11.038318
max_paid_inv_0_12m                 11.038318
max_paid_inv_0_24m                 11.038318
name_in_email                      11.038318
active_div_by_paid_ratio_0_12m     29.933286
num_active_invoices                11.038318
num_arch_dc_0_12m                  11.038318
num_arch_dc_12_24m                 11.038318
num_arch_ok_0_12m                  11.038318
num_arch_ok_12_24m                 11.038318
num_arch_rem_0_12m                 11.038318
max_archived_status_0_6_months     11.038318
max_archived_status_0_12_months    11.038318
max_archived_status_0_24_months    11.038318
recovery_debt                      11.038318
sum_capital_paid_acct_0_12m        11.038318
sum_capital_paid_acct_12_24m       11.038318
sum_paid_inv_0_12m                 11.038318
time_hours                         11.038318
dtype: float64
```

Dropping the missing values which is more than 49% data and the updated shape of the dataset is (99979, 29)

Dropping the variables (worst_status_12_24m, worst_status_6_12m, debt_vs_paid_ratio_0_24m, worst_status_3_6m, worst_status_0_3m, account_status, avg_payment_span_0_3m)

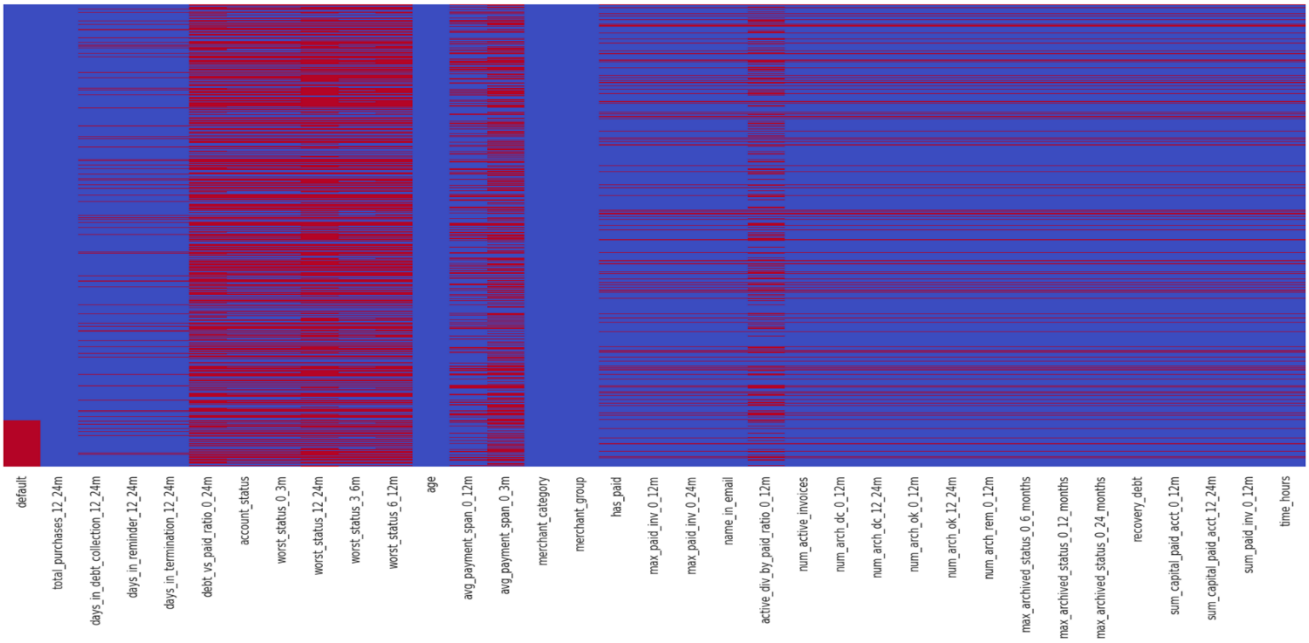

Figure 8: Missing values representation in heat map

Now treating the remaining missing values with mean for all numerical values and mode for categorical values and output is mentioned below

```
default                          99979
total_purchases_12_24m           99979
days_in_debt_collection_12_24m   99979
days_in_reminder_12_24m          99979
days_in_termination_12_24m       99979
age                              99979
avg_payment_span_0_12m           99979
avg_payment_span_0_3m            99979
merchant_category                99979
merchant_group                   99979
has_paid                         99979
max_paid_inv_0_12m               99979
max_paid_inv_0_24m               99979
name_in_email                    99979
active_div_by_paid_ratio_0_12m   99979
num_active_invoices              99979
num_arch_dc_0_12m                99979
num_arch_dc_12_24m               99979
num_arch_ok_0_12m                99979
num_arch_ok_12_24m               99979
num_arch_rem_0_12m               99979
max_archived_status_0_6_months   99979
max_archived_status_0_12_months  99979
max_archived_status_0_24_months  99979
recovery_debt                    99979
sum_capital_paid_acct_0_12m      99979
sum_capital_paid_acct_12_24m     99979
sum_paid_inv_0_12m               99979
time_hours                       99979
```

Outlier detection – Boxplot

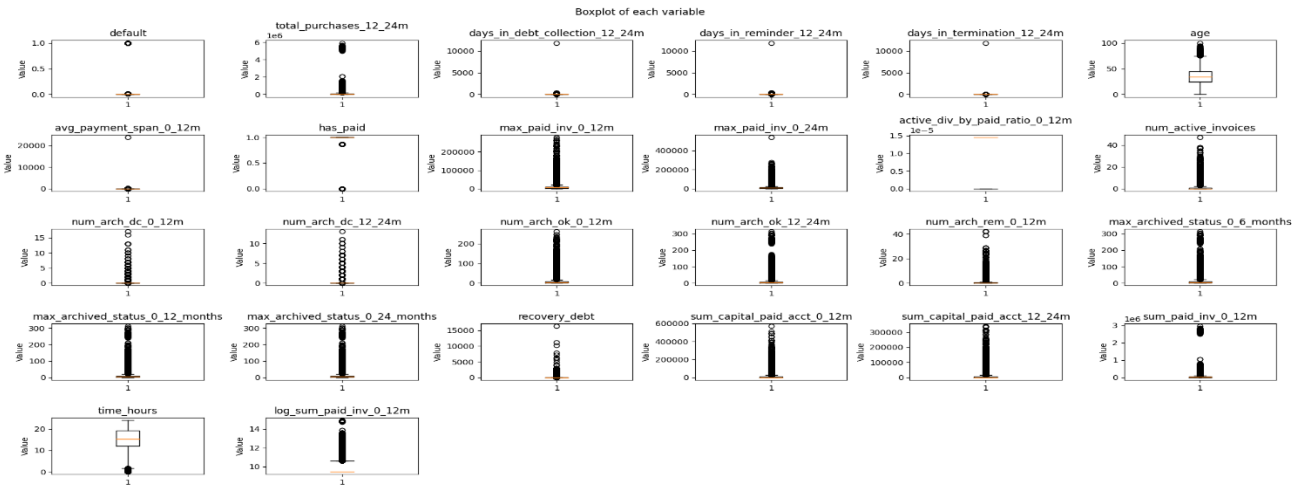

Figure 9: Outlier representation in boxplots for numerical variables

1. Approach involves detection of outliers in each numerical variables in the dataset using Z-score (setting the threshold value as 3) based outlier detection technique.
2. Winsorization technique is used to treat the outliers

```
Number of outliers in each column:
default                              1289
total_purchases_12_24m               1228
days_in_debt_collection_12_24m         52
days_in_reminder_12_24m               662
days_in_termination_12_24m              1
age                                   433
avg_payment_span_0_12m                  2
has_paid                                0
max_paid_inv_0_12m                   1687
max_paid_inv_0_24m                   1687
active_div_by_paid_ratio_0_12m          0
num_active_invoices                  1287
num_arch_dc_0_12m                     960
num_arch_dc_12_24m                    961
num_arch_ok_0_12m                    2051
num_arch_ok_12_24m                   1809
num_arch_rem_0_12m                   1508
max_archived_status_0_6_months       2037
max_archived_status_0_12_months      2037
max_archived_status_0_24_months      2037
recovery_debt                         192
sum_capital_paid_acct_0_12m          2151
sum_capital_paid_acct_12_24m         2166
sum_paid_inv_0_12m                   1228
time_hours                           1067
log_sum_paid_inv_0_12m                  0
```

```
Updated count of outliers after treatment:
 active_div_by_paid_ratio_0_12m        0
age                                    0
avg_payment_span_0_12m                 0
days_in_debt_collection_12_24m         0
days_in_reminder_12_24m                0
days_in_termination_12_24m             0
default                             1289
has_paid                               0
log_sum_paid_inv_0_12m                 0
max_archived_status_0_12_months        0
max_archived_status_0_24_months        0
max_archived_status_0_6_months         0
max_paid_inv_0_12m                     0
max_paid_inv_0_24m                     0
merchant_category                      0
merchant_group                         0
name_in_email                          0
num_active_invoices                    0
num_arch_dc_0_12m                      0
num_arch_dc_12_24m                     0
num_arch_ok_0_12m                      0
num_arch_ok_12_24m                     0
num_arch_rem_0_12m                     0
recovery_debt                          0
sum_capital_paid_acct_0_12m            0
sum_capital_paid_acct_12_24m           0
sum_paid_inv_0_12m                     0
time_hours                             0
total_purchases_12_24m                 0
```

- Post treating the outliers some of the rows is replaced with (inf and –inf) which is dropped from the treated dataset for model training and the updated shape of the dataset is (71096, 29)
- Default is selected as the target variable
- Numerical feature data is standardized before training the model

3 b. Need for variable transformation (if any)

Sol: We have transformed the variable from 'log_sum_paid_inv_0_12m' to this 'sum_paid_inv_0_12m' by applying the log to the variable.

3 c. Variables removed or added and why (if any)

Sol: We found that variable userid which is not required. By applying the drop function, we removed the userid column and the below is the updated output head of the dataset.

	default	total_purchases_12_24m	days_in_debt_collection_12_24m	days_in_reminder_12_24m	days_in_termination_12_24m	debt_vs_paid_ratio_0_24m	account_status
0	0.0	0.0	0.0	0.0	0.0	0.0	1.0
1	0.0	0.0	0.0	0.0	0.0	NaN	1.0
2	0.0	0.0	0.0	0.0	0.0	NaN	NaN
3	0.0	0.0	NaN	NaN	NaN	NaN	NaN
4	0.0	0.0	0.0	0.0	0.0	NaN	NaN

Also, further we will remove the data while treating the missing values

1.Total Purchases in the Last 12-24 Months: The line `df['total_purchases_12_24m'] = df['sum_paid_inv_0_12m'] + df['sum_paid_inv_0_12m']` calculates the total amount spent by the customer on purchases using the credit card in the last 12-24 months.

This variable is useful for understanding the overall spending behavior of customers over a specific period.

2. Active Divisions by Paid Ratio in the Last 0-12 Months: The line `df['active_div_by_paid_ratio_0_12m'] = df['num_active_invoices'] / df['sum_paid_inv_0_12m']` computes the ratio of the number of active invoices to the total amount paid by the customer in the last 0-12 months.

This variable provides insights into how actively customers are making payments relative to the number of invoices they have.

3. Maximum Archived Status in the Last 0-6 Months: The line `df['max_archived_status_0_6_months'] = df[['num_arch_dc_0_12m', 'num_arch_dc_12_24m', 'num_arch_ok_0_12m', 'num_arch_ok_12_24m', 'num_arch_rem_0_12m']].max(axis=1)` determines the maximum archived status among different categories (debt collection, OK, and reminder) in the last 0-6 months.

This variable helps identify the most severe status a customer account has been in over a specific period, providing insights into their payment behavior.

4. Maximum Archived Status in the Last 0-12 Months and 0-24 Months: The lines `df['max_archived_status_0_12_months'] = df[['max_archived_status_0_6_months', 'max_archived_status_0_6_months']].max(axis=1)` and `df['max_archived_status_0_24_months'] = df[['max_archived_status_0_12_months', 'max_archived_status_0_12_months']].max(axis=1)` extend the calculation of maximum archived status to the last 0-12 months and 0-24 months, respectively.

These variables provide a broader perspective on the historical account status and help in assessing customer risk over longer periods.

4. Model building

4 a. Clear on why was a particular model(s) chosen and improve model performance

Sol: Predicting default probabilities is pivotal for prudent decision-making in credit card operations. This project endeavors to construct a robust predictive model tailored for a credit card company, aiming to forecast default likelihoods by leveraging diverse customer attributes. The chosen models, ranging from Logistic Regression to advanced ensemble techniques like Gradient Boosting, are meticulously selected to encapsulate the intricacies of credit risk assessment. By integrating cutting-edge methodologies, the study seeks to empower the company with actionable insights, fortifying its resilience against potential default risks and promoting sustainable growth within the dynamic credit card banking industry.

- Post treating the outliers some of the rows is replaced with (inf and –inf) which is dropped from the treated dataset for model training and the updated shape of the dataset is (71096, 29)
- Default is selected as the target variable
- Numerical feature data is standardized before training the model
- Categorical data is transformed into numerical data using the 'OneHotEncoder' technique for modeling

Now the data is split in train and test data (70:30)

The shape of the dataset is (71096, 29) after completing all the data cleaning step including transforming the categorical data using 'OneHotEncoder' technique for model training and testing.

Before initiating the model building the dataset is standardised and spitted into Train and test dataset i.e. 70: 30 in ration.

Output of train and test dataset:

```
Training data shape (X_train, y_train): (56876, 28) (56876,)
Testing data shape (X_test, y_test): (14220, 28) (14220,)
```

Now we are building the below mentioned models for the banking dataset:

1. Logistic Regression Model
2. Decision Tree Model
3. Random Forest Model
4. AdaBoost Model
5. Gradient Boosting Model
6. Bagging Model
7. LightGBM Model
8. XGBoost Model
9. Linear Discriminant Analysis Model (LDA)

Training results summary for each model along with the graphs

1. Logistic Regression:

As per the results the model achieved and 98.99% accuracy which will be further compared with all the models.

```
Model: Logistic Regression
Accuracy: 0.9899198274649538
Classification Report:
              precision    recall  f1-score   support

         0.0       0.99      1.00      0.99     21113
         1.0       1.00      0.00      0.01       216

    accuracy                           0.99     21329
   macro avg       0.99      0.50      0.50     21329
weighted avg       0.99      0.99      0.98     21329
```

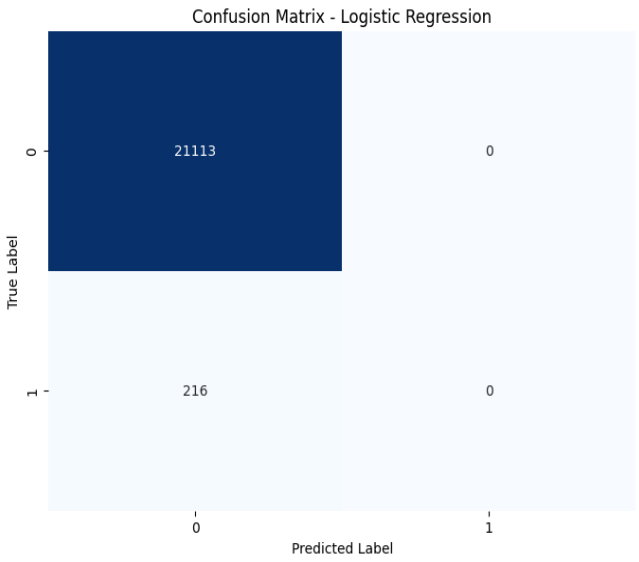

Figure 10: Train - Confusion Matrix – Logistic regression

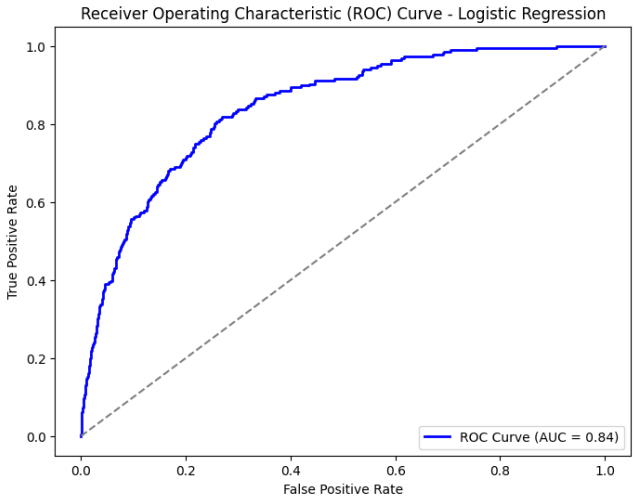

Figure 11: Train - (ROC) Curve – Logistic regression

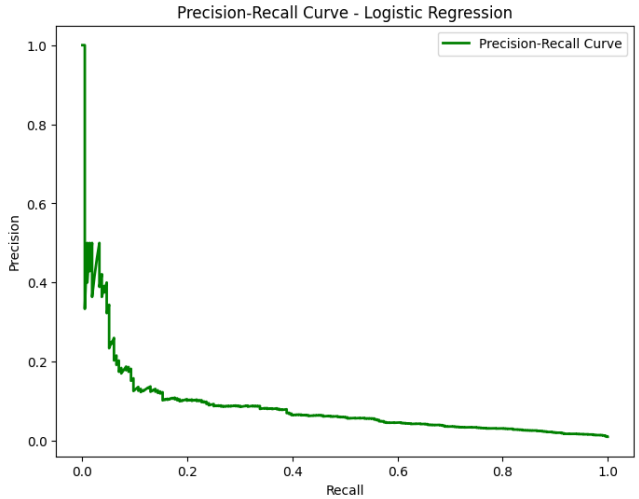

Figure 12: Train - Precision-Recall Curve – Logistic regression

2. Decision Tree Model

As per the results the model achieved and 98.11% accuracy and classification report output:

```
Model: Decision Tree
Accuracy: 0.9811524215856345
Classification Report:
              precision    recall  f1-score   support

         0.0       0.99      0.99      0.99     21113
         1.0       0.14      0.17      0.16       216

    accuracy                           0.98     21329
   macro avg       0.57      0.58      0.57     21329
weighted avg       0.98      0.98      0.98     21329
```

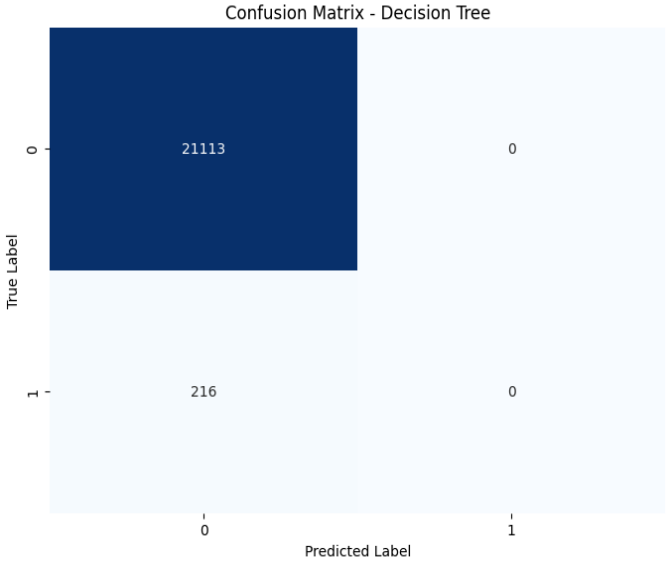

Figure 13: Train - Confusion Matrix – Decision Tree

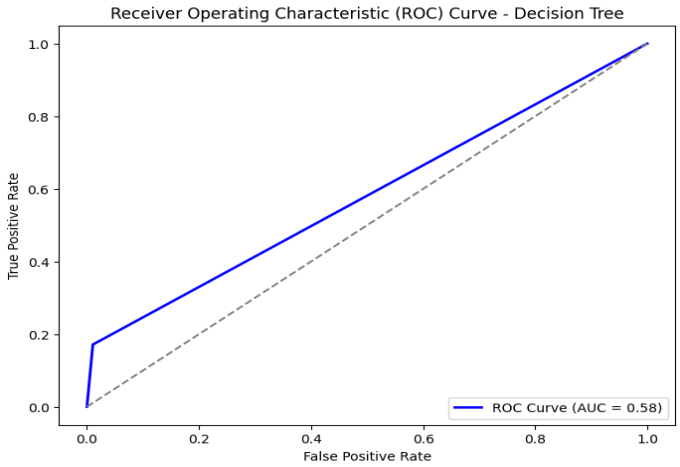

Figure 14: Train - (ROC) Curve – Decision tree Tree

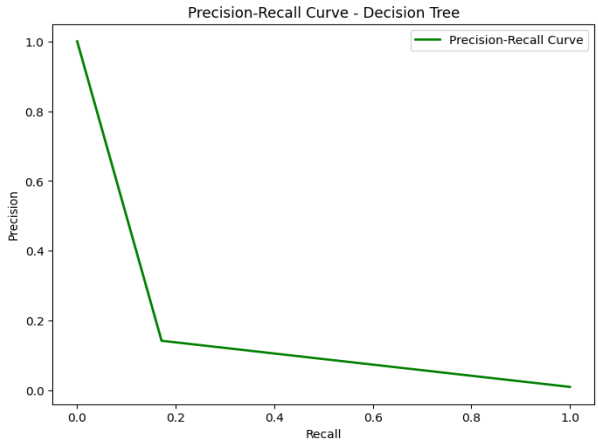

Figure 15: Train - Precision-Recall Curve – Decision

3. Random Forest Model

As per the results the model achieved and 99.034% accuracy and classification report output:

```
Model: Random Forest
Accuracy: 0.9903417881757232
Classification Report:
              precision    recall  f1-score   support

         0.0       0.99      1.00      1.00     21113
         1.0       0.86      0.06      0.10       216

    accuracy                           0.99     21329
   macro avg       0.92      0.53      0.55     21329
weighted avg       0.99      0.99      0.99     21329
```

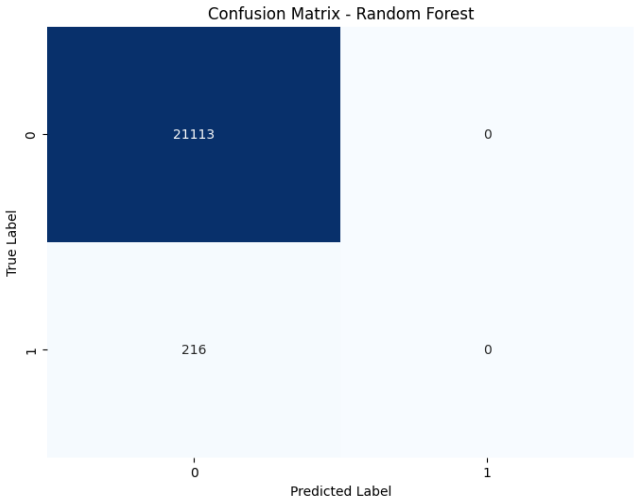

Figure 16: Train - Confusion Matrix – Random Forest

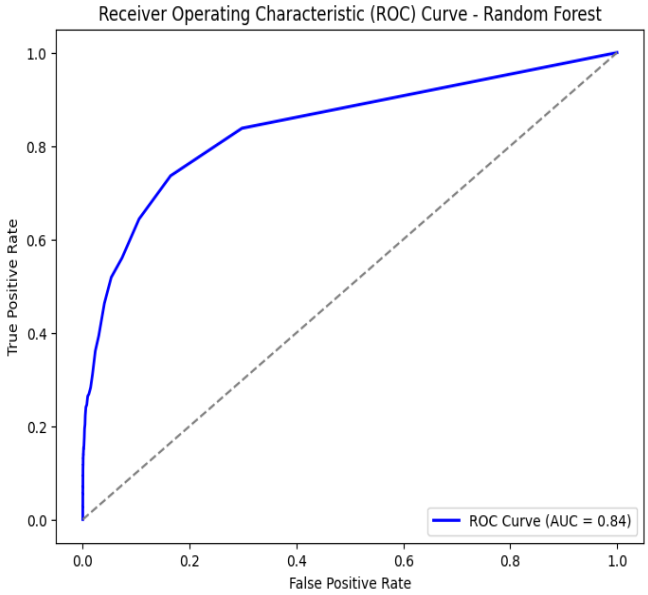

Figure 17: Train – ROC – Random Forest

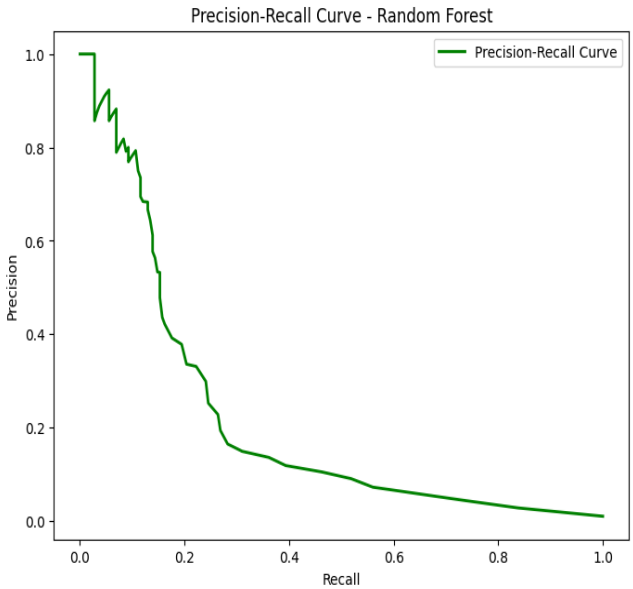

Figure 18: Train - Precision-Recall Curve – Random Forest

4. AdaBoost Model

As per the results the model achieved and 98.95% accuracy and classification report output:

```
Model: AdaBoost
Accuracy: 0.9895447512776032
Classification Report:
              precision    recall  f1-score   support

         0.0       0.99      1.00      0.99     21113
         1.0       0.33      0.03      0.06       216

    accuracy                           0.99     21329
   macro avg       0.66      0.52      0.53     21329
weighted avg       0.98      0.99      0.99     21329
```

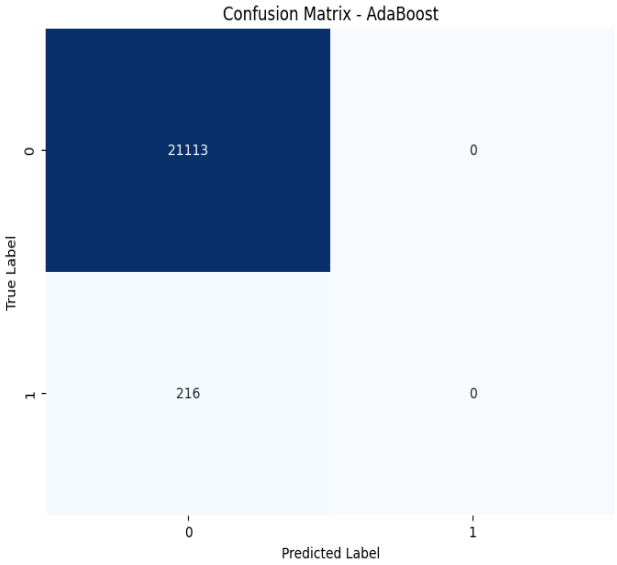

Figure 19: Train - Confusion Matrix – AdaBoost

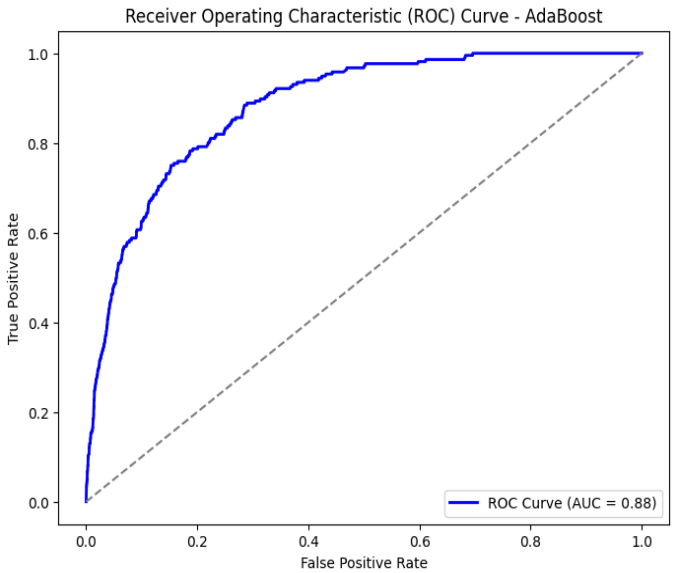

Figure 20: Train - (ROC) Curve – AdaBoost

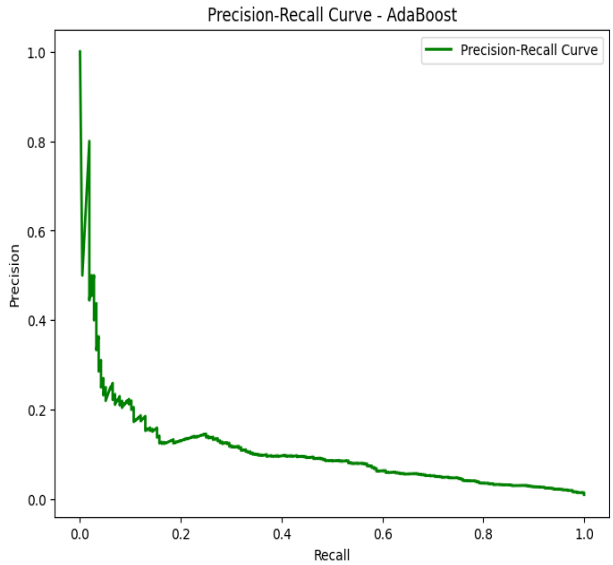

Figure 21: Train - Precision-Recall Curve – AdaBoost

5. Gradient Boosting Model

As per the results the model achieved and 98.90% accuracy and classification report output:

```
Model: Gradient Boosting
Accuracy: 0.9890759060434151
Classification Report:
              precision    recall  f1-score   support

         0.0       0.99      1.00      0.99     21113
         1.0       0.26      0.04      0.07       216

    accuracy                           0.99     21329
   macro avg       0.62      0.52      0.53     21329
weighted avg       0.98      0.99      0.99     21329
```

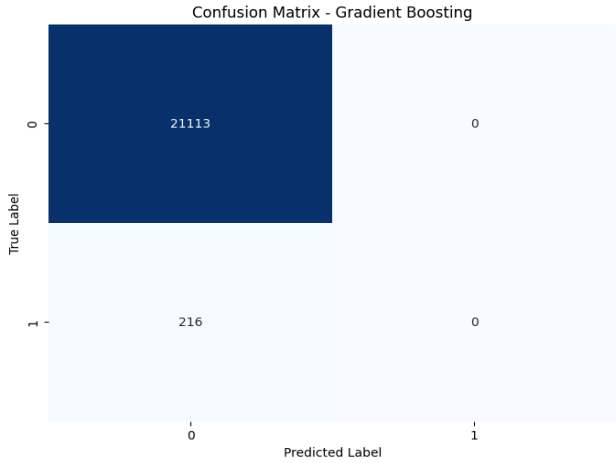

Figure 22: Train - Confusion Matrix – Gradient Boosting

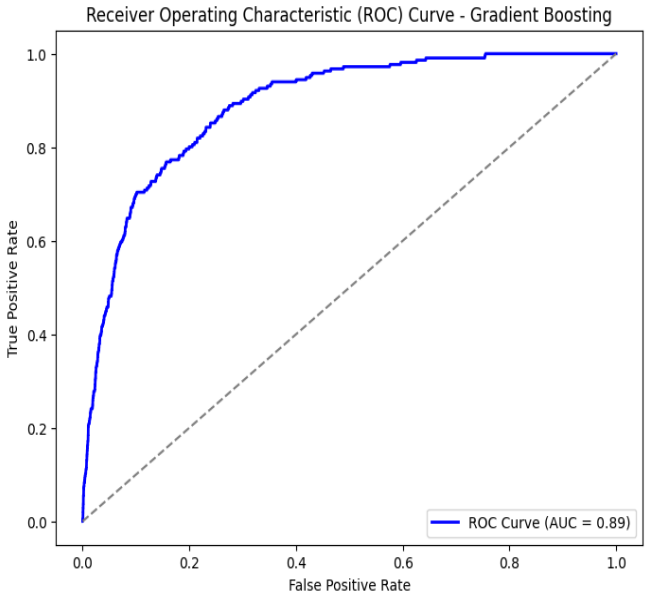

Figure 23: Train - (ROC) Curve – Gradient Boosting

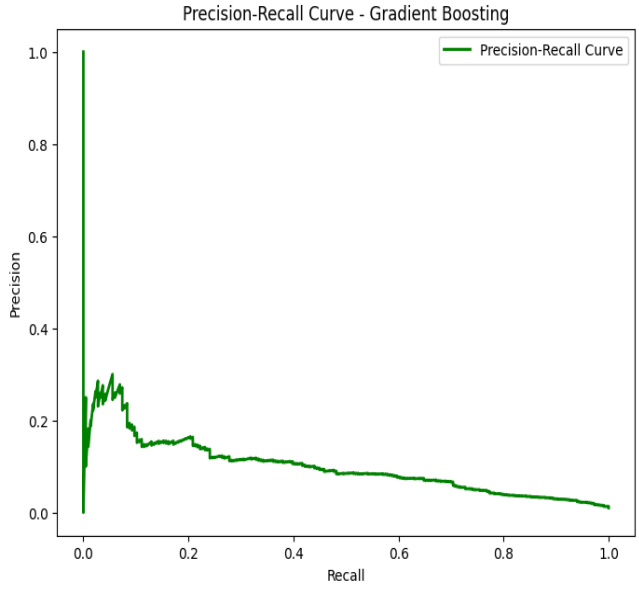

Figure 24: Train - Precision-Recall Curve – Gradient Boosting

6. Bagging Model

As per the results the model achieved and 98.99% accuracy and classification report output:

```
Model: Bagging
Accuracy: 0.9899198274649538
Classification Report:
              precision    recall  f1-score   support

         0.0       0.99      1.00      0.99     21113
         1.0       0.53      0.04      0.08       216

    accuracy                           0.99     21329
   macro avg       0.76      0.52      0.54     21329
weighted avg       0.99      0.99      0.99     21329
```

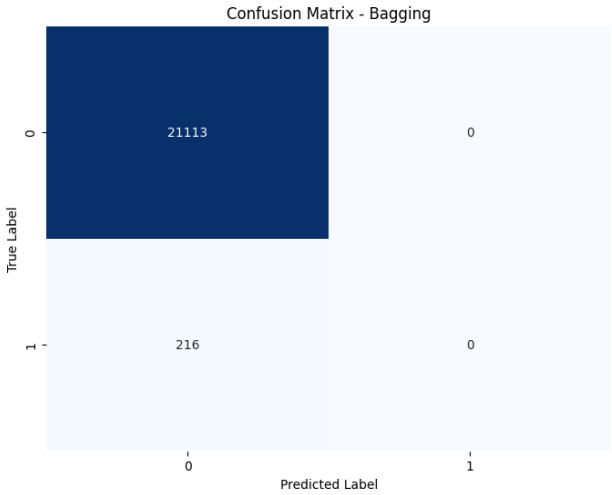

Figure 25: Train - Confusion Matrix – Bagging

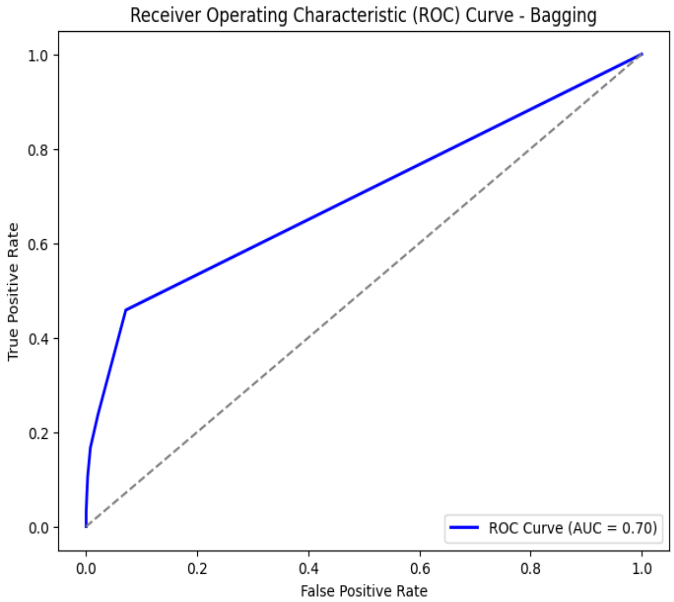

Figure 26: Train - (ROC) Curve – Bagging

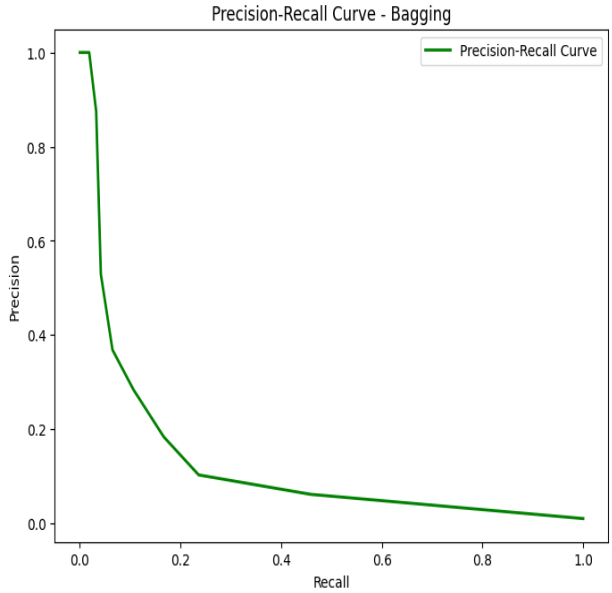

Figure 27: Train - Precision-Recall Curve – Bagging

7. Light GBM Model

As per the results the model achieved and 98.90% accuracy and classification report output:

```
Model: LightGBM
Accuracy: 0.9890759060434151
Classification Report:
              precision    recall  f1-score   support

         0.0       0.99      1.00      0.99     21113
         1.0       0.33      0.07      0.12       216

    accuracy                           0.99     21329
   macro avg       0.66      0.54      0.56     21329
weighted avg       0.98      0.99      0.99     21329
```

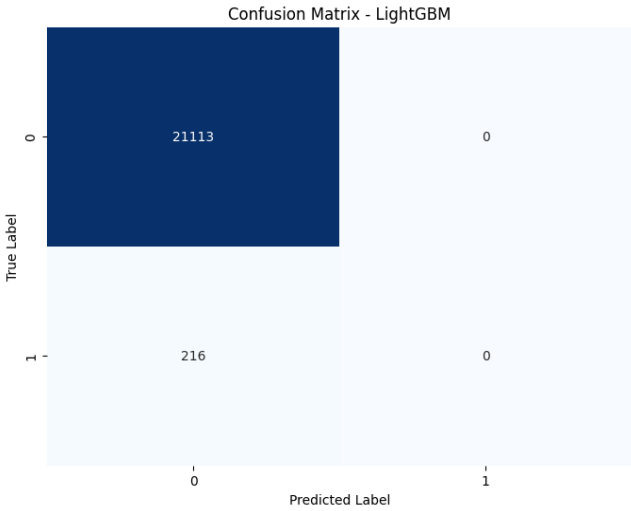

Figure 28: Train - Confusion Matrix – Light GBM

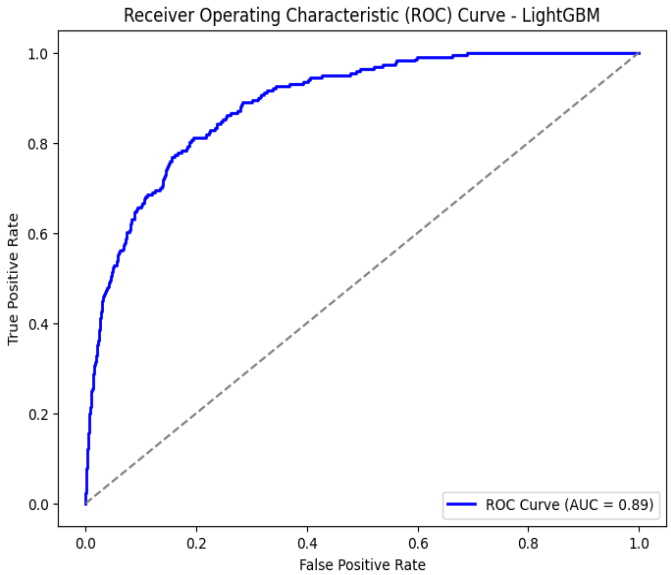

Figure 29: Train - (ROC) Curve – Light GBM

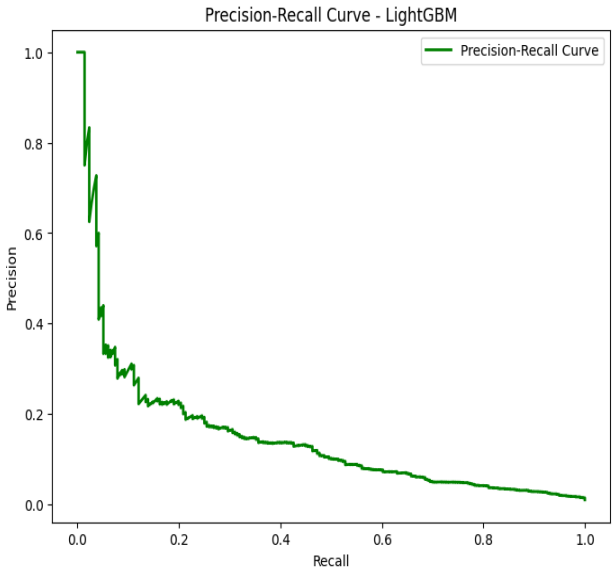

Figure 30: Train - Precision-Recall Curve – Light GBM

8. XG Boost Model

As per the results the model achieved and 98.90% accuracy and classification report output:

```
Model: XGBoost
Accuracy: 0.9900604810352103
Classification Report:
              precision    recall  f1-score   support

         0.0       0.99      1.00      1.00     21113
         1.0       0.58      0.07      0.12       216

    accuracy                           0.99     21329
   macro avg       0.78      0.53      0.56     21329
weighted avg       0.99      0.99      0.99     21329
```

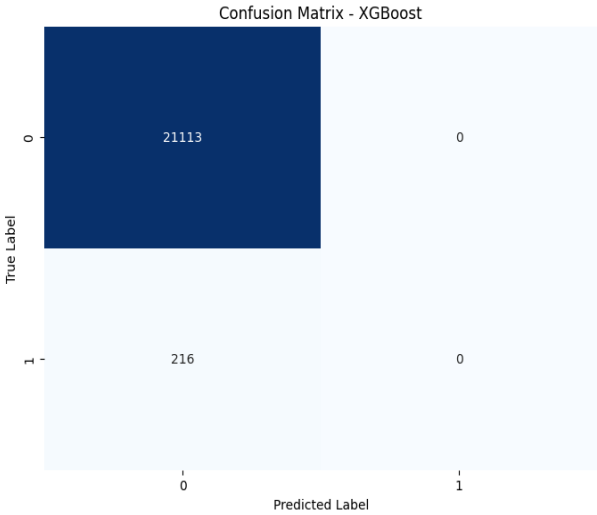

Figure 31: Train - Confusion Matrix – XG Boost

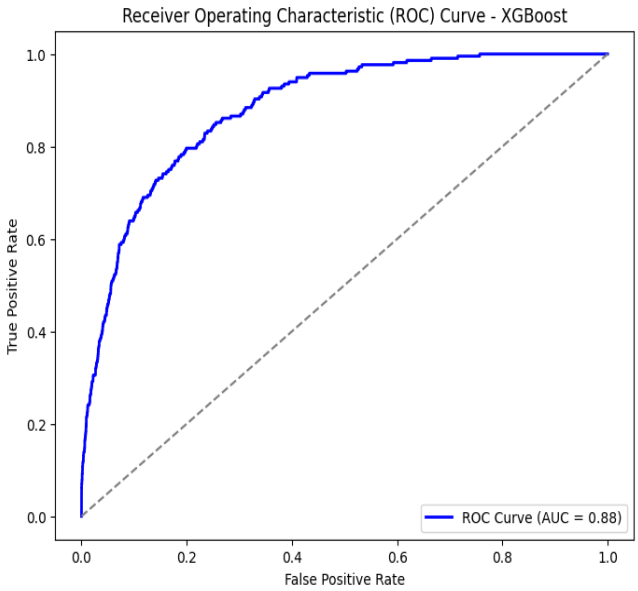

Figure 32: Train - (ROC) Curve – XG Boost

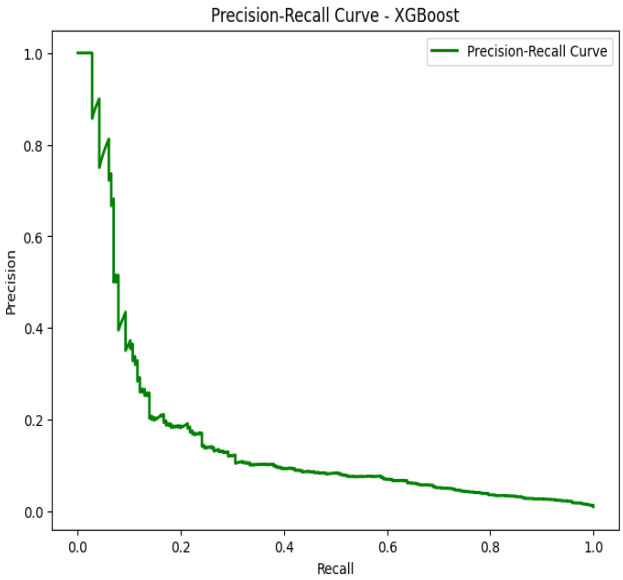

Figure 33: Train - Precision-Recall Curve – XG Boost

9. Linear Discriminant Analysis Model (LDA)

As per the results the model achieved and 98.19% accuracy and classification report output:

```
Model: Linear Discriminant Analysis
Accuracy: 0.9819494584837545
Classification Report:
              precision    recall  f1-score   support

         0.0       0.99      0.99      0.99     21113
         1.0       0.10      0.09      0.09       216

    accuracy                           0.98     21329
   macro avg       0.54      0.54      0.54     21329
weighted avg       0.98      0.98      0.98     21329
```

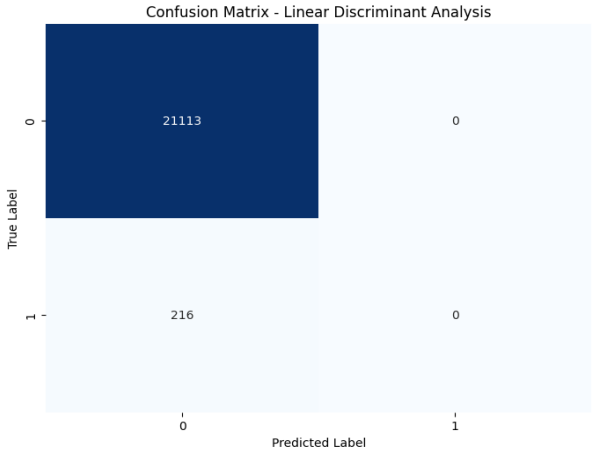

Figure 34: Train - Confusion Matrix – LDA

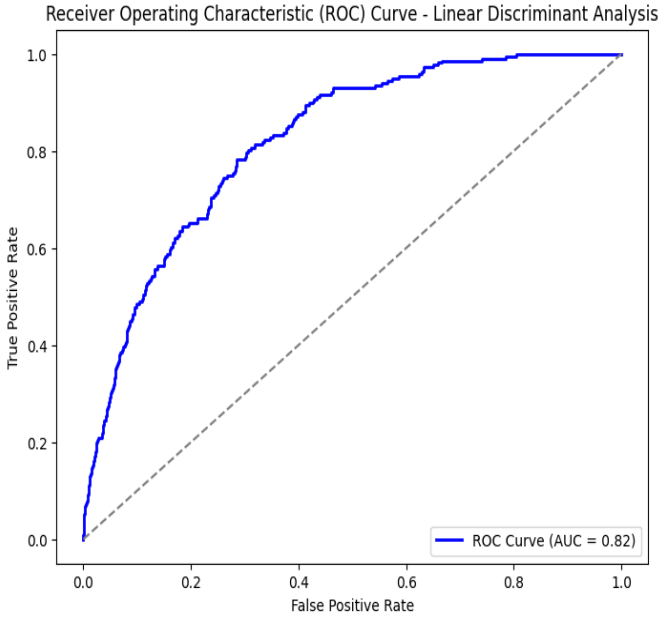

Figure 35: Train - (ROC) Curve – XG Boost

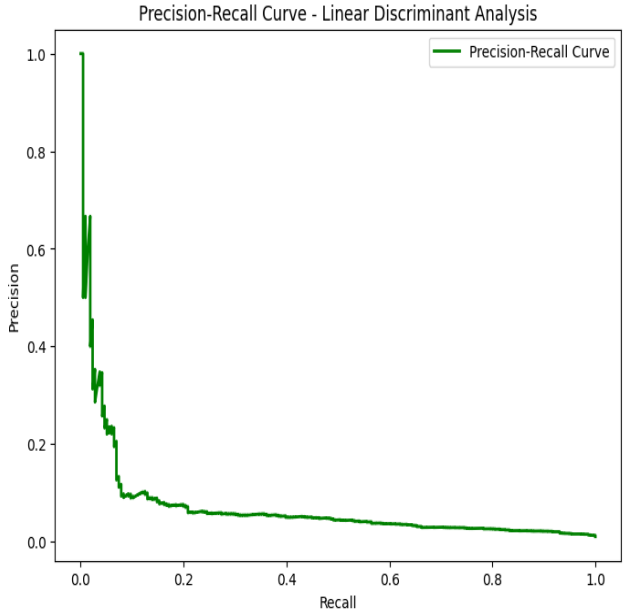

Figure 36: Train - Precision-Recall Curve – XG Boost

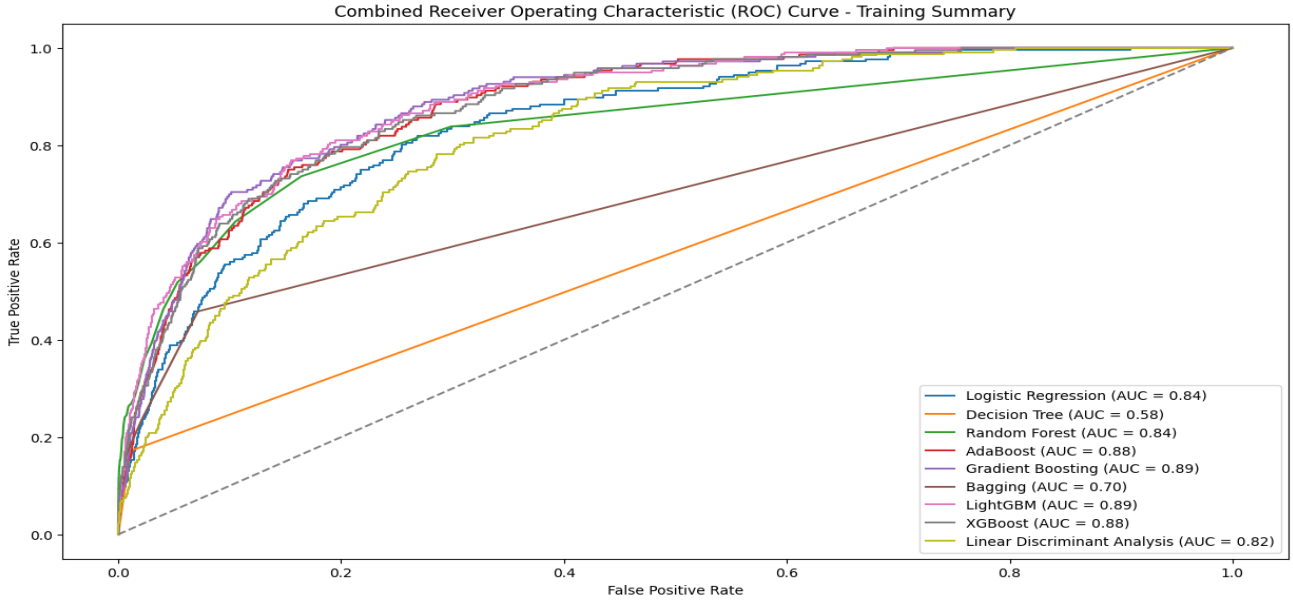

Figure 37: Train summary – Combined (ROC) Curve for all models

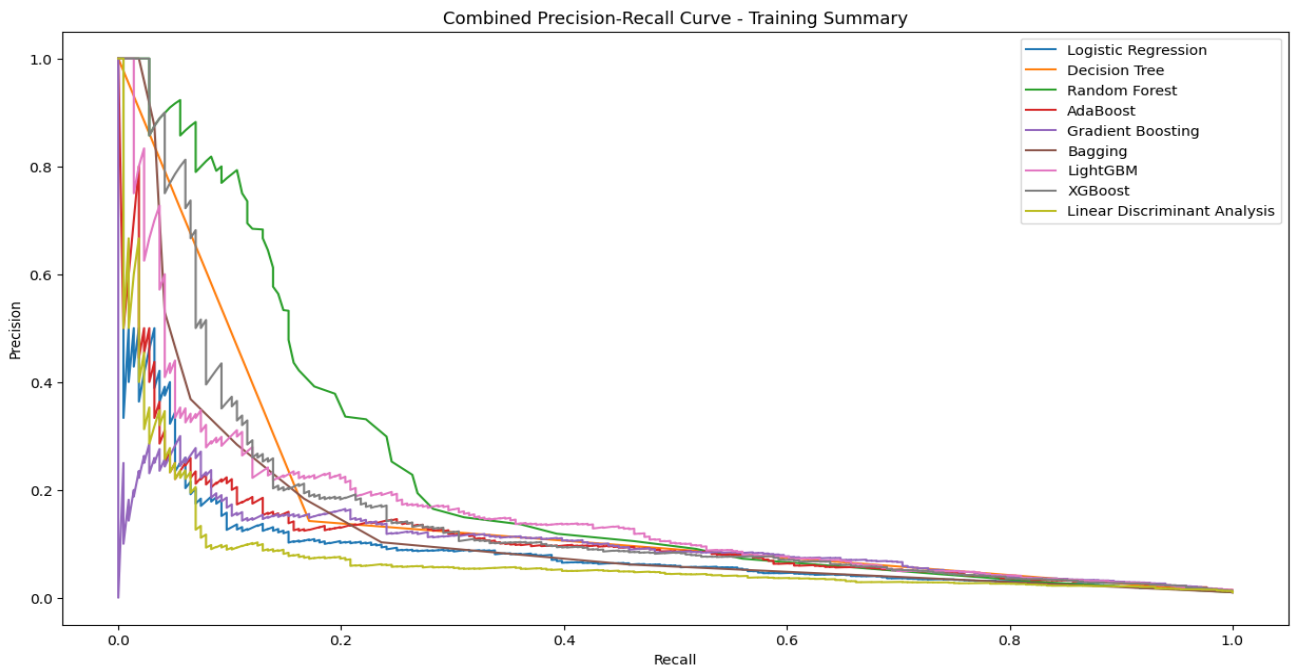

Figure 38: Training summary – Combined Precision-Recall Curve for all models

5. Model validation

5a. How was the model validated? Just accuracy, or anything else too?

Sol: Test result summary of all the models and confusion matrix results are same as training results

1. **Logistic Regression – 98.99%**

```
Test Results Summary for Model: Logistic Regression
Test Accuracy: 0.9899198274649538
Test Classification Report:
              precision    recall  f1-score   support

         0.0       0.99      1.00      0.99     21113
         1.0       1.00      0.00      0.01       216

    accuracy                           0.99     21329
   macro avg       0.99      0.50      0.50     21329
weighted avg       0.99      0.99      0.98     21329
```

2. **Decision Tree – 98.11%**

```
Test Results Summary for Model: Decision Tree
Test Accuracy: 0.9811524215856345
Test Classification Report:
              precision    recall  f1-score   support

         0.0       0.99      0.99      0.99     21113
         1.0       0.14      0.17      0.16       216

    accuracy                           0.98     21329
   macro avg       0.57      0.58      0.57     21329
weighted avg       0.98      0.98      0.98     21329
```

3. **Random forest – 99.03%**

```
Test Results Summary for Model: Random Forest
Test Accuracy: 0.9903417881757232
Test Classification Report:
              precision    recall  f1-score   support

         0.0       0.99      1.00      1.00     21113
         1.0       0.86      0.06      0.10       216

    accuracy                           0.99     21329
   macro avg       0.92      0.53      0.55     21329
weighted avg       0.99      0.99      0.99     21329
```

4. **Ada Boost - 98.95%**

```
Test Results Summary for Model: AdaBoost
Test Accuracy: 0.9895447512776032
Test Classification Report:
              precision    recall  f1-score   support

         0.0       0.99      1.00      0.99     21113
         1.0       0.33      0.03      0.06       216

    accuracy                           0.99     21329
   macro avg       0.66      0.52      0.53     21329
weighted avg       0.98      0.99      0.99     21329
```

5. **Gradient Boosting – 98.90%**

```
Test Results Summary for Model: Gradient Boosting
Test Accuracy: 0.9890759060434151
Test Classification Report:
              precision    recall  f1-score   support

         0.0       0.99      1.00      0.99     21113
         1.0       0.26      0.04      0.07       216

    accuracy                           0.99     21329
   macro avg       0.62      0.52      0.53     21329
weighted avg       0.98      0.99      0.99     21329
```

6. **Bagging Model - 98.99%**

```
Test Results Summary for Model: Bagging
Test Accuracy: 0.9899198274649538
Test Classification Report:
              precision    recall  f1-score   support

         0.0       0.99      1.00      0.99     21113
         1.0       0.53      0.04      0.08       216

    accuracy                           0.99     21329
   macro avg       0.76      0.52      0.54     21329
weighted avg       0.99      0.99      0.99     21329
```

7. **Light GBM – 98.90%**

```
Test Results Summary for Model: LightGBM
Test Accuracy: 0.9890759060434151
Test Classification Report:
              precision    recall  f1-score   support

         0.0       0.99      1.00      0.99     21113
         1.0       0.33      0.07      0.12       216

    accuracy                           0.99     21329
   macro avg       0.66      0.54      0.56     21329
weighted avg       0.98      0.99      0.99     21329
```

8. **XGBoost – 99.00%**

```
Test Results Summary for Model: XGBoost
Test Accuracy: 0.9900604810352103
Test Classification Report:
              precision    recall  f1-score   support

         0.0       0.99      1.00      1.00     21113
         1.0       0.58      0.07      0.12       216

    accuracy                           0.99     21329
   macro avg       0.78      0.53      0.56     21329
weighted avg       0.99      0.99      0.99     21329
```

9. **Linear Discriminant Analysis – 98.19%**

```
Test Results Summary for Model: Linear Discriminant Analysis
Test Accuracy: 0.9819494584837545
Test Classification Report:
              precision    recall  f1-score   support

         0.0       0.99      0.99      0.99     21113
         1.0       0.10      0.09      0.09       216

    accuracy                           0.98     21329
   macro avg       0.54      0.54      0.54     21329
weighted avg       0.98      0.98      0.98     21329
```

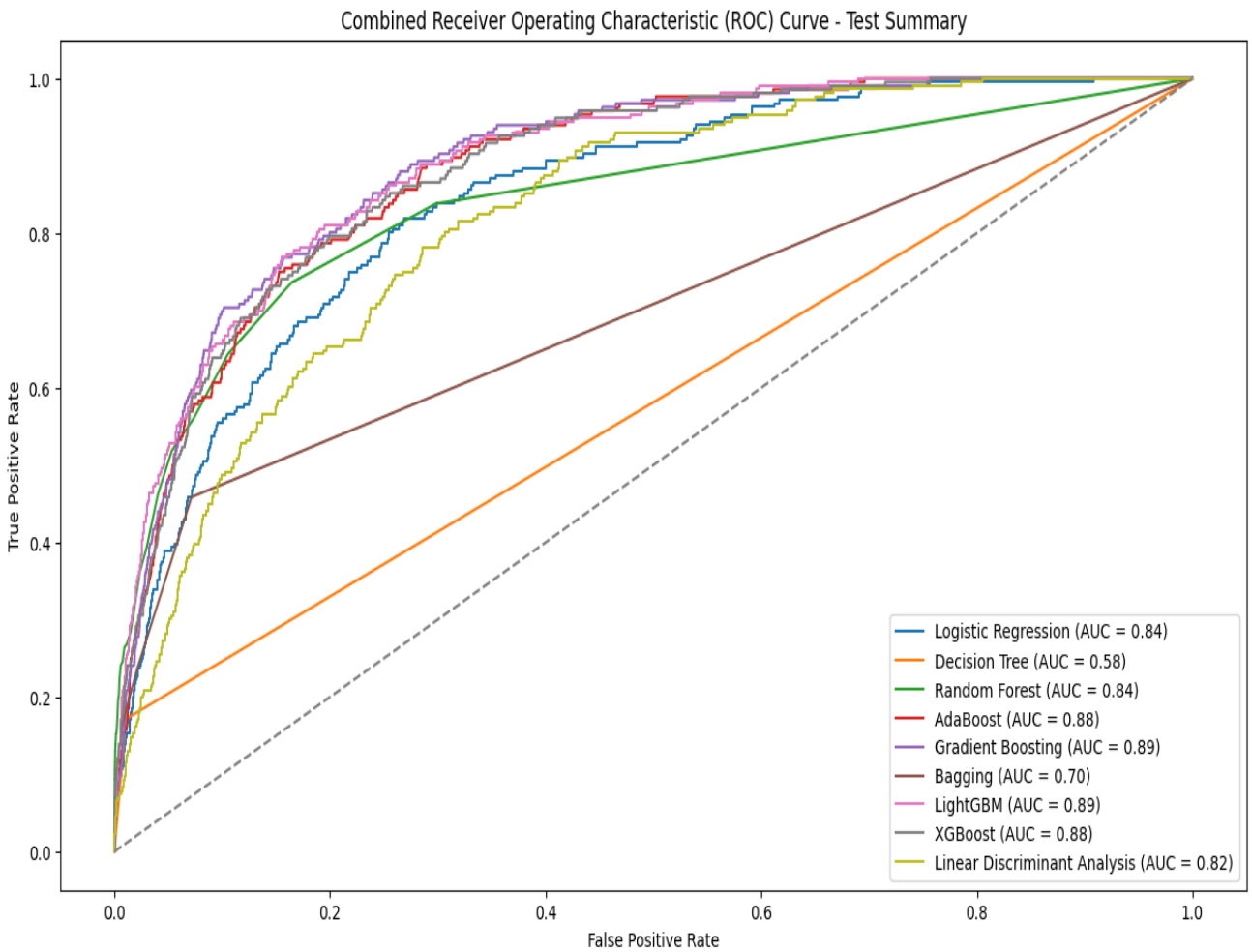

Figure 39: Test summary – Combined (ROC) Curve for all models

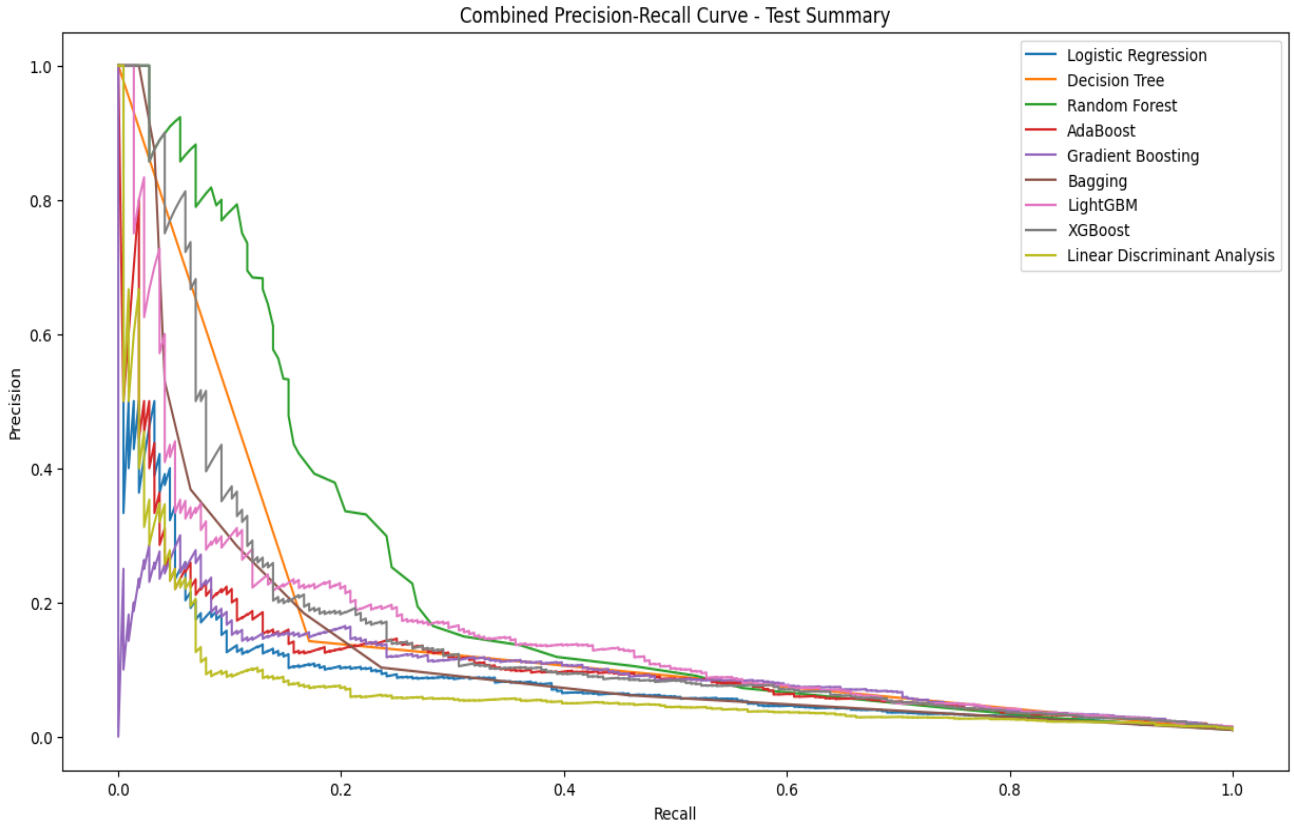

Figure 40: Test summary – Combined Precision-Recall Curve for all models

- Post testing all the models and from the results, Random_forest classifier is the best model with test accuracy - 0.9904355572225
- Applying GridsearchCV for model tuning to random forest model
- From the results it has been observed the post model tuning the test accuracy is decreased to 0.9902949036523043
- In this case, the difference in accuracy is minimal, indicates that the Random Forest model was performing very well and not required significant tuning.

```
Random Forest Model Tuning Results:
Best Hyperparameters: {'max_depth': None, 'n_estimators': 50}
Best Model Accuracy: 0.9902949036523043
Classification Report:
              precision    recall  f1-score   support

         0.0       0.99      1.00      1.00     21113
         1.0       0.85      0.05      0.10       216

    accuracy                           0.99     21329
   macro avg       0.92      0.53      0.55     21329
weighted avg       0.99      0.99      0.99     21329
```

6. Detailed recommendations for the management/client based on the analysis done

S. No	Name of the model	Train Accuracy	Test Accuracy
1	Random Forest Classifier Model	99.034	99.034
2	XGB Classifier Model	99.006	99.006
3	Logistic Regression Model	98.991	98.991
4	AdaBoost Classifier Model	98.954	98.954
5	Bagging Classifier Model	98.991	98.991
6	Gradient Boosting Classifier Model	98.907	98.907
7	Light GBM Classifier Model	98.907	98.907
8	Linear Discriminant Analysis Model	98.194	98.194
9	Decision Tree Classifier Model	98.115	98.115

Table 1: Model comparison – Train and Test Accuracies

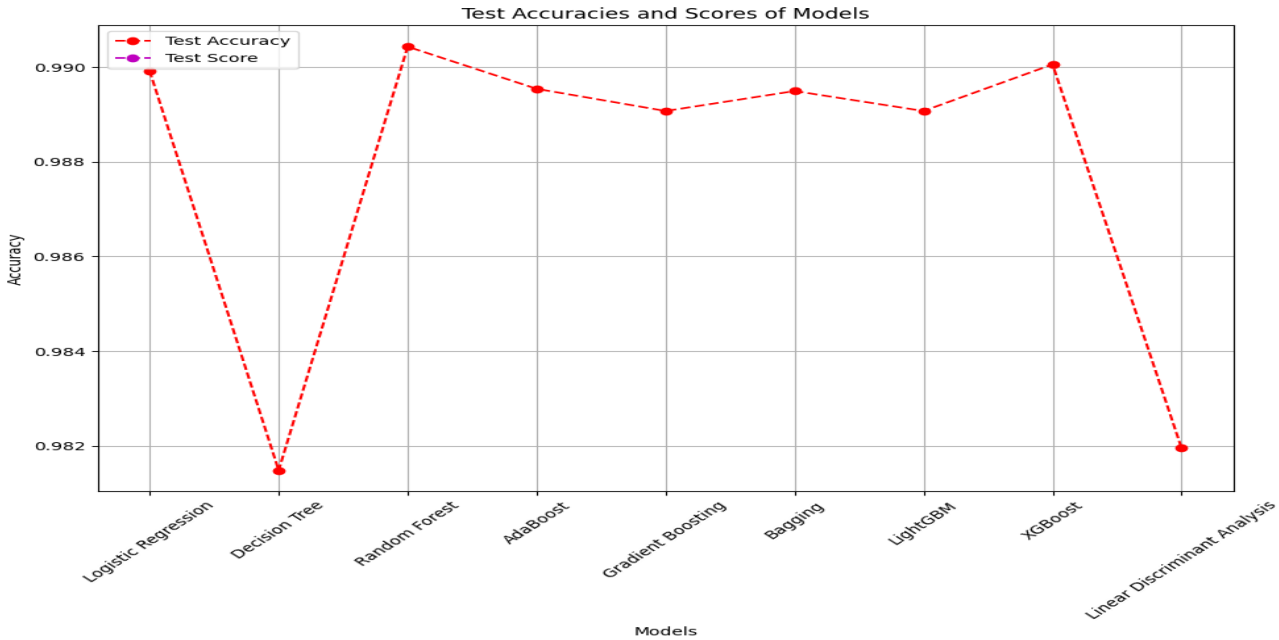

Figure 41: Test accuracy comparison for all models

The evaluation results of the classification models provide valuable insights into their performance in predicting the probability of default for the credit card company. Here's the interpretation of the models' performance based on the provided test results summary:

1. **Logistic Regression:** Accuracy: 98.99%
   - Precision for class 0.0 (non-defaulters): 99%, indicating accurate identification of customers who will not default.
   - Precision for class 1.0 (defaulters): 100%, but recall is 0%, suggesting the model fails to identify most instances of defaulters.
   - F1-score for class 1.0 is very low at 0.01, indicating poor balance between precision and recall for defaulters.
2. **Decision Tree:** Accuracy: 98.12%
   - Precision for class 0.0: 99%, indicating accurate identification of non-defaulters.
   - Precision for class 1.0: 14%, with recall at 17%, suggesting the model's ability to identify defaulters is limited.
3. **Random Forest:** Accuracy: 99.03%
   - Precision for class 0.0: 99%, indicating accurate identification of non-defaulters.
   - Precision for class 1.0: 86%, with recall at 6%, indicating some ability to identify defaulters but with limitations.
4. **AdaBoost:** Accuracy: 98.95%
   - Precision for class 0.0: 99%, indicating accurate identification of non-defaulters.
   - Precision for class 1.0: 33%, with recall at 3%, suggesting limited ability to identify defaulters.
5. **Gradient Boosting:** Accuracy: 98.91%
   - Precision for class 0.0: 99%, indicating accurate identification of non-defaulters.
   - Precision for class 1.0: 26%, with recall at 4%, indicating limited ability to identify defaulters.
6. **Bagging:** Accuracy: 98.99%
   - Precision for class 0.0: 99%, indicating accurate identification of non-defaulters.
   - Precision for class 1.0: 53%, with recall at 4%, suggesting some ability to identify defaulters but with limitations.
7. **Light GBM:** Accuracy: 98.91%
   - Precision for class 0.0: 99%, indicating accurate identification of non-defaulters.
   - Precision for class 1.0: 33%, with recall at 7%, indicating limited ability to identify defaulters.
8. **XG Boost**: Accuracy: 99.01%
   - Precision for class 0.0: 99%, indicating accurate identification of non-defaulters.
   - Precision for class 1.0: 58%, with recall at 7%, suggesting some ability to identify defaulters but with limitations.
9. **Linear Discriminant Analysis:** Accuracy: 98.19%
   - Precision for class 0.0: 99%, indicating accurate identification of non-defaulters.
   - Precision for class 1.0: 10%, with recall at 9%, suggesting limited ability to identify defaulters.

Overall, while some models demonstrate high accuracy and precision for non-defaulters, their performance in identifying defaulters is generally poor, with low recall rates. This indicates a need for further model refinement and possibly feature engineering to improve the predictive capability of the models for default prediction.

Business recommendations to the client:

Based on the comprehensive analysis of various predictive models for default prediction in credit card operations, several key recommendations for the management/client:

1. **Utilize Predictive Models:** Embrace the integration of predictive models into risk management practices to enhance the accuracy and efficiency of default prediction. Leveraging machine learning algorithms offers a systematic approach to assess credit risk and make informed decisions.
2. **Improve Credit Assessment**: Harness insights derived from predictive models to refine credit assessment processes. By incorporating model predictions into decision-making frameworks, the company can optimize credit allocation strategies and minimize default risks.
3. **Efficient Resource Allocation:** Utilize predictive model outputs to allocate resources judiciously, focusing on customers with higher default probabilities. By directing resources towards high-risk segments, the company can maximize profitability while minimizing exposure to default-related losses.
4. **Proactive Customer Protection:** Implement proactive measures aimed at mitigating default risks and safeguarding customers' financial well-being. By offering tailored assistance and support to at-risk customers, the company can foster trust and loyalty while minimizing default occurrences.
5. **Enhance Competitiveness:** Present proactive risk management strategies as a key differentiator in the competitive landscape. By demonstrating a commitment to customer protection and financial prudence, the company can strengthen its market position and attract discerning customers.

In summary, the analysis underscores the importance of integrating predictive analytics into credit card operations to enhance risk management practices, improve decision-making, and ultimately safeguard the financial health and reputation of the company. By adopting a proactive approach to default prediction and mitigation, the company can position itself as a leader in responsible lending practices while maximizing profitability and ensuring long-term sustainability.